

GX Series

모션 프로그램 매뉴얼



CSCAM

문서번호 : **CS-GX245-MP-26-01**

Contents

Contents	3
1 프로그램	1-1
1.1 프로그램 파일	1-2
1.2 프로그램 요소	1-3
블록	1-3
워드	1-4
어드레스	1-4
데이터	1-6
1.3 프로그램 진행	1-7
주석문	1-7
문 번호	1-7
1.4 메인 프로그램과 서브 프로그램	1-9
서브 프로그램	1-10
서브 프로그램 호출 다중도	1-10
서브 프로그램 호출과 복귀	1-10
메인 프로그램 반복	1-12
2 모션 언어	2-1
2.1 모션 명령어	2-2
2.2 연산 명령	2-4
3 좌표계	3-1
3.1 좌표계	3-2
기계 좌표계	3-2
작업물 좌표계	3-2
3.2 기계 좌표계	3-3
3.3 작업물 좌표계 설정	3-4
3.4 평면 선택	3-6
3.5 임의의 평면 선택	3-8
4 좌표치와 치수	4-1
4.1 절대 지령	4-2
4.2 증분 지령	4-3
4.3 Inch / Metric 단위 입력	4-4
5 모션 기능	5-1
5.1 위치 결정	5-2
5.2 직선 보간	5-4

5.3	원호 보간.....	5-6
5.4	헬리컬 보간	5-11
5.5	스킵 기능.....	5-13
5.6	Teaching 기능.....	5-14
5.7	1 원점 이송	5-16
5.8	2, 3, 4 원점 이송.....	5-17
5.9	휴지	5-18
5.10	Single-Block.....	5-19
5.11	EOM.....	5-20
5.12	신호 Wait.....	5-21
5.13	프로그램 종료	5-22
6	이송 기능	6-1
6.1	위치결정의 이송속도 지령	6-2
6.2	보간 이송의 속도 지령	6-4
7	제어 및 연산 명령	7-1
7.1	제어 명령.....	7-2
	무조건 분기 (GOTO 문).....	7-2
	조건 분기(IF 문).....	7-2
	조건 분기(SWITCH 문).....	7-3
	반복(WHILE 문)	7-5
	반복(FOR 문)	7-6
7.2	연산 명령.....	7-10
	변수의 정수, 치환	7-10
	가법형 연산	7-10
	승법형 연산	7-10
	연산의 조합	7-10
	수학 함수	7-10
	Round 사용법	7-11
	BCD to BINARY.....	7-12
	BINARY to BCD.....	7-12
	Bit Right Shift	7-13
	Bit Left Shift	7-14
	Move Bit(MOV B)	7-16
	Move Data(MOV D)	7-18
	Set Bit(SET B)	7-19
	Set Data(SET D)	7-20

8 보조 기능	8-1
8.1 일반적인 M 코드.....	8-2
8.2 코드에 의한 서브 프로그램 호출.....	8-3
8.3 동기 M 코드 설정	8-4
8.4 S Code 기능.....	8-6
9 매크로 프로그램	9-1
9.1 매크로 호출 명령어.....	9-2
단순 호출	9-2
ML 코드에 의한 매크로 호출.....	9-6
SCALL과 MCALL의 차이점	9-7
다중 호출	9-8
9.2 커스텀 매크로 본체의 작성	9-11
매크로 프로그램 본체의 형식	9-11
변수의 사용방법	9-11
9.3 프로그램 예제	9-13
10 매크로 프로그램	1
10.1 매크로 호출 명령	2
10.1.1 빠른 명령	2
10.1.2 ML 코드에 의한 매크로 호출.....	6
10.1.3 G코드에 의한 매크로 호출.....	6
10.1.4 M코드에 의한 매크로 호출	7
10.1.5 SCALL 과 MCALL의 차이점	8
10.1.6 다중 호출	8
10.2 커스텀 매크로 본체 준비	11
10.2.1 매크로 프로그램 본체의 형식	11
10.2.2 변수 사용 방법	11
10.3 프로그램 예제	13

Preface

Conventions

본 매뉴얼에는 이해를 돕기 위해서 아래와 같은 기호 표시가 있습니다. 이 표시는 **GX-Builder** 소프트웨어의 조작에 관련된 내용과 **GX** 시스템 설정 및 동작 시에 주의 사항 또는 참고 사항을 나타내는 내용에 표시되어 있습니다.



[주의]

주의사항을 나타내는 표시입니다. 이 표시와 같이 설명된 내용은 시스템에 치명적인 오작동을 유발할 수 있으므로 반드시 숙지하시기 바랍니다.



[Tip]

참고사항을 나타내는 표시입니다.



GX-Builder 소프트웨어에 있는 조작 버튼을 나타냅니다. **GX-Builder**에는 많은 조작 버튼들이 있습니다. 이 조작 버튼의 사용법 및 기능을 설명할 때 와 같이 표시합니다.

[메뉴]

GX-Builder 소프트웨어에 있는 메뉴를 나타냅니다. 메뉴는 각 항목에 하위 메뉴가 있을 수도 있으며, 이 경우 '➔' 기호로 구분하여 제일 상위 메뉴부터 차례대로 표시합니다. (예. [파일 ➔ 저장]) 또한, 해당 위치에서 마우스의 오른쪽 버튼을 누르면 컨텍스트 메뉴(빠른 실행 메뉴)가 실행됩니다. 이때도 마찬가지로 위와 같은 형식으로 표시됩니다. (예. [프로젝트저장])

1

프로그램

1.1 프로그램 파일

프로그램 내용을 가진 파일을 프로그램 파일이라고 하고 “ 프로그램이름.확장자 ”의 형태로 저장됩니다. 프로그램 이름 형식은 오직 숫자(4자리)로만 작성해야 합니다. 또한, 확장자는 “ mp ”입니다.

프로그램의 용량은 기본적으로 제한이 없으나 사용하는 GX 시리즈의 저장매체 용량에 제한을 받습니다.

1.2 프로그램 요소

프로그램은 GX 시리즈가 인식할 수 있는 언어로 작성되며 일련의 동작지령을 나타내는 블록들이 모여서 구성됩니다.

블록은 워드의 모임으로 구성되고 워드는 어드레스와 데이터로 구성됩니다.

\$, &, ', :, ?, @, W, |, ^, _, {, }, ~, 키보드에서 Del 키 등과 같이 GX에서 정의되지 않은 문자를 지령한 경우에는 알람이 발생합니다.

이벤트 코드	이벤트 내용
331	정의되지 않는 문자가 존재합니다.

블록

기계를 구동하기 위한 기본 지령 단위로서 1개의 워드 또는 여러 개 워드들의 조합으로 구성됩니다. 한 블록은 프로그램의 한 줄에 해당됩니다.

한 블록에 올 수 있는 문자의 최대 개수는 300개입니다. 300개의 문자에는 공백도 포함됩니다.

NOO	GOO	XO.O	YO.O	ZO.O	MOO	SOOO	FOO
문 번호 지령	준비지령	좌표 지령			M Code	S Code	속도지령

만약, 이 제한을 넘게 지령하면 알람이 발생합니다.

프로그램 실행 시에 같은 블록에서는 각 워드의 순서는 상관 없습니다. 통상적으로 위와 같은 순서로 입력합니다. 한 블록의 실행이 완료되면 다음 블록으로 진행하게 됩니다.

이벤트 코드	이벤트 내용
369	하나의 블록 문자 개수는 300개로 제한 됩니다.

워드

워드는 프로그램의 기능 명령의 기본 단위이며 어드레스와 데이터로 구성됩니다. 데이터는 어드레스 다음에 바로 이어서 위치합니다.

	적용 예제
--	-------

```
N100 ;
MOV( G00 ) ;
X100 ;
```

한 블록에서 같은 내용의 워드를 두개 이상 지령하면 앞에 지령 한 것은 무시되고 뒤에 지령 된 워드가 수행됩니다.

	적용 예제
--	-------

```
MOV( G00 ) X100 X200 ;           - X100은 무시되고 X200이 실행된다.
```

어드레스

어드레스는 기본 어드레스와 특수 어드레스가 있습니다.

기본 어드레스는 알파벳 A ~ Z 중에 1개로 정의되고 지령에 대한 의미를 알려주는 기능을 수행합니다.

특수 어드레스는 주석문인 ;, “ 있습니다.

	적용 예제	
파라미터	설정값	내용
P1100	00000000 - 00000000 - 00001111 - 11111111	축 사용 여부 설정
P1105	00000000 - 00000000 - 00000000 - 00000000	축 형태
P900	00000000 - 00000000 - 00000000 - 00000011	제어 채널 사용 여부 선택
P921	00000000 - 00000000 - 00000000 - 00000111	채널별 축 선택(1 채널)
P922	00000000 - 00000000 - 00000000 - 00111000	채널별 축 선택(2 채널)
P941	1 - (X)	채널별 축 이름에 해당하는 축 번호 설정(1 채널)
P942	2 - (Y)	
P943	3 - (Z)	
P944	0 - (A)	

P945	0 - (B)	
P946	0 - (C)	
P947	0 - (U)	
P948	0 - (V)	
P949	0 - (W)	
P951	4 - (X)	채널별 축 이름에 해당하는 축 번호 설정(2 채널)
P952	5 - (Y)	
P953	6 - (Z)	
P954	0 - (A)	
P955	0 - (B)	
P956	0 - (C)	
P957	0 - (U)	
P958	0 - (V)	
P959	0 - (W)	

채널 1 - X100 Y100 Z100 위치로 이송속도 1500으로 급속이송 방법으로 이동.

채널 2 - X-100 Y-100 Z-100 위치로 이송속도 3000으로 급속이송 방법으로 이동을 모션 프로그램으로 작성하면 다음과 같다.

채널1	일반	X100 Y100 Z100 F1500 ;
	Axis 코드	A1=100 A2=100 A3=100 F1500 ;
채널2	일반	X-100 Y-100 Z-100 F3000 ;
	Axis 코드	A1=-100 A2=-100 A3=-100 F3000 ;

Axis 코드를 작성할 때 채널 1을 제외한 나머지 채널에서는 축 번호와 상관없이 P222에 설정된 순서대로 A1, A2, A3, 형식으로 번호를 부여한다.

위의 표에서 보면 채널 2는 4, 5, 6축을 설정을 했지만 모션 프로그램에서는 A1, A2, A3으로 사용하는 것을 볼 수 있다.

또한 각 축에 대한 이송속도를 적용할 때에도 위의 내용과 같이 이루어지는데 채널 2에서 각각 1500, 2000, 2500으로 이송속도를 적용한다면 다음과 같이 할 수 있다.

채널2	일반	X-100 Y-100 Z-100 F1=1500 F2=2000 F3=2500 ;
	Axis 코드	A1=-100 A2=-100 A3=-100 F1=1500 F2=2000 F3=2500 ;

이송속도에 대한 자세한 내용은 7장. 이송 기능 참조.

데이터

데이터는 어드레스 지령 수행 시에 필요한 값을 알려주는 기능을 수행합니다.

모든 데이터 지령은 부호를 제외하고 12자 이내로 제한됩니다.

만약, 이 제한을 넘는 지령을 하게 되면 알람이 발생합니다.

실수 형태로 지령 되는 데이터에서 소수점을 한 개 이상 지령하게 되면 알람이 발생 합니다.

이벤트 코드	이벤트 내용
332	숫자가 최대 버퍼를 넘었습니다.
333	소수점이 한 개 이상입니다.

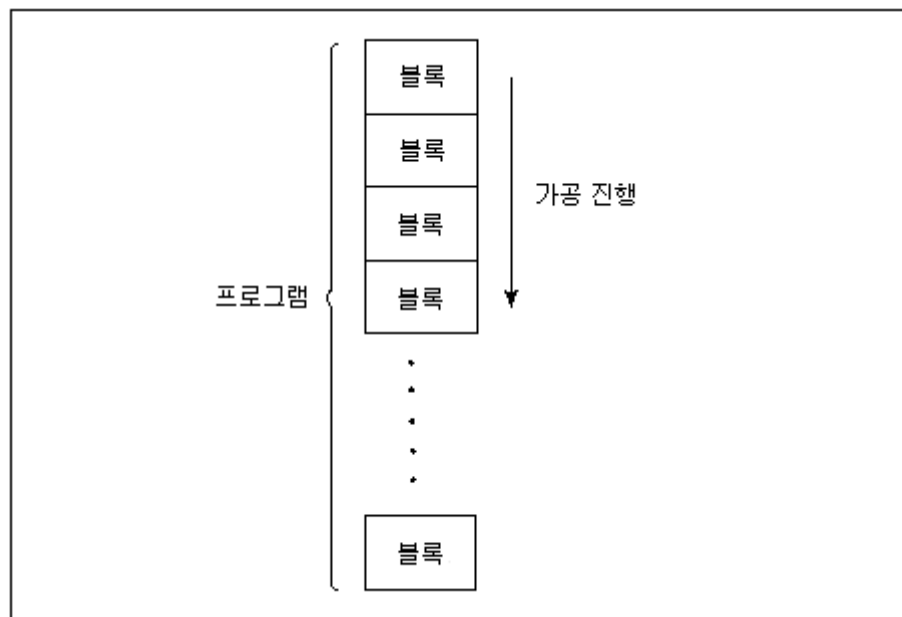
1.3 프로그램 진행

프로그램은 블록들의 조합으로 구성되며 블록의 내용에 따라서 해당되는 기능이 수행됩니다. 주로 첫 블록에는 “ 1234 ; 와 같이 프로그램 파일명을 표기하고 주석 기능을 사용하여 프로그램 설명을 작성합니다.

“, ‘, ‘ 로 시작하는 블록에 다른 지령을 하게 되면 무시됩니다.

블록의 맨 마지막에는 프로그램 종료 서브 지령인 **PEND**가 있어야 합니다.

만약 **PEND** 지령을 하지 않게 되면 알람이 발생합니다.



이벤트 코드	이벤트 내용
363	종료지령 없이 종료하였습니다.

주석문

주석으로 처리되는 명령에는 ;과 “ 가 있습니다,
;과 “ 명령은 지령이 된 곳부터 뒤의 모든 블록 내용이 주석 처리됩니다.

문 번호

블록의 선두에 **N** 어드레스와 수치로 지령하는 명령입니다.

문 번호는 특정 블록으로 프로그램의 수행을 옮기기 위해 사용됩니다. 번호는 순서

에 관계없이 지정할 수 있으나 프로그램 해석을 용이하게 하기 위해서는 상위 블록에서 하위 블록으로 순차적으로 지정하는 것이 좋습니다.

여러 블록에 동일한 문 번호를 지령하게 되면 알람이 발생합니다.

Nxx에서 **N** 다음에 수치가 없이 **N**만 지령 된 경우에도 문 번호로 인식합니다.

문 번호 지령은 번호와 크기에 상관없이 100개를 사용할 수 있습니다.

이벤트 코드	이벤트 내용
349	동일한 진행블록이 존재합니다.

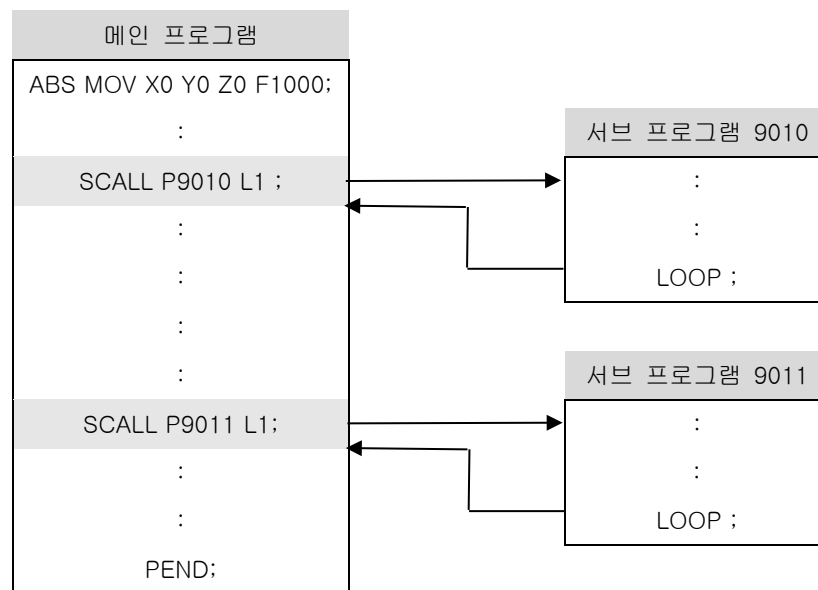
1.4 메인 프로그램과 서브 프로그램

프로그램은 메인 프로그램과 서브 프로그램으로 나누어 집니다.

메인 프로그램은 정지 상태에서부터 이송을 최초로 시작하는 프로그램입니다.

서브 프로그램은 주 프로그램의 호출에 의해 실행되고 실행이 종료되면 메인 프로그램으로 복귀합니다.

실행 프로그램은 메인 프로그램 단독으로 구성될 수 있고 또한 메인 프로그램과 여러 서브 프로그램으로 구성됩니다.



서브 프로그램

이송할 형상이 반복되거나 패턴이 일정한 동작을 반복하는 경우와 같이 프로그램 중에 자주 쓰이는 형상을 프로그램 해놓고 다른 프로그램에서 불러 쓰도록 구성할 수 있습니다. 이렇게 프로그램을 활용하기 위해서 반복되는 이송 부분을 하나의 프로그램으로 미리 작성하여 놓고 필요 시 호출하여 사용하는 기능을 서브 프로그램 호출이라 하고 이 때 호출되는 프로그램을 서브 프로그램이라 합니다. 서브 프로그램은 4자리 숫자로만 이름을 만들 수 있습니다.

주의	서브 프로그램 이름은 4 자리 숫자로만 가능합니다.
----	------------------------------

서브 프로그램 호출 다중도

메인 프로그램이 서브 프로그램을 호출할 수 있고 서브 프로그램은 또 다른 서브 프로그램을 호출할 수 있습니다. 한 번 서브 프로그램을 호출하는 것을 1중이라고 할 때 최고 4 중까지 호출 가능합니다.

만약, 부 프로그램 호출 횟수가 4중이 넘게 되면 알람이 발생합니다.

이벤트 코드	이벤트 내용
353	최대 부프로그램 호출을 초과하였습니다.

서브 프로그램 호출과 복귀

[프로그램 입력 형식]

```
SCALL( M98 ) P_ Q_ R_ L_ ;
LOOP( M99 ) P_ ;
```

[프로그램 입력 설명]

SCALL(M98)	- 서브 프로그램 호출
LOOP(M99)	- 서브 프로그램 종료
P_	- SCALL(M98)인 경우 서브 프로그램 이름(_는 4자리 숫자)
Q_	- SCALL(M98)인 경우 복귀해야 할 블록의 문 번호 (생략하면 첫 블록부터 수행)
R_	- LOOP(M99)인 경우 복귀해야 할 블록의 문 번호

(생략하면 LOOP(M99)가 나올 때까지 진행)

L_ - 서브 프로그램 호출 반복 횟수

서브 프로그램 호출 시에 문법이 맞지 않으면 알람이 발생합니다.

호출된 서브 프로그램의 마지막 블록은 **LOOP**를 단독으로 사용해야 합니다.

만약, 호출 시 **R_** 지령을 사용하지 않은 상황에서 서브 프로그램에서 **LOOP**를 지령하지 않으면 알람이 발생하고 **LOOP** 지령 시 문법이 맞지 않으면 알람이 발생합니다. 1중 이상으로 연속해서 서브 프로그램을 호출 시에 이미 앞에서 호출된 프로그램을 다시 호출하게 되면 서브 프로그램 호출이 무한 루프에 빠지게 됩니다. 이와 같이 이미 호출한 서브 프로그램을 다시 호출하게 되면, 알람이 발생합니다.

이벤트 코드	이벤트 내용
352	부프로그램 호출 문법이 맞지 않습니다.
354	이미 호출된 프로그램입니다.
355	부프로그램에 반복지령이 없습니다.
356	반복지령 문법이 맞지 않습니다.

적용 예제

“ 메인 프로그램 - 서브 프로그램 호출

MOV(G00) X0 Y0 Z0 ;

X20 Y40 ;

SCALL(M98) P2222 Q1 R2 L2 ;

- 서브 프로그램(2222) 호출, 2회 반복

X70 Y20 ;

SCALL(M98) P2222 ;

- 서브 프로그램(2222) 호출

INC(G91) REF(G28) X0 Y0;

PEND ;

- 메인 프로그램 종료

“ 서브 프로그램 - 2222.mp

N1 ABS(G90) MOV(G00) Z5 ;

LIN(G01) Z-10 F20 ;

Z-25 F100 ;

N2 MOV(G00) Z50 ;

LOOP(M99) ;

메인 프로그램 반복

[프로그램 입력 형식]

LOOP(M99) P_ L_ ;

[프로그램 입력 설명]

LOOP(M99)	- 메인 프로그램 반복
P_	- 반복 시 시작 블록 문 번호
L_	- 반복 횟수

메인 프로그램에서 **LOOP**를 지령하면 반복 기능으로 수행됩니다. **P_**를 지령하지 않으면 메인 프로그램의 첫 블록부터 반복 수행되며 **L_**를 지령하지 않으면 무한하게 반복 수행 됩니다.

적용 예제

```

“ 메인 프로그램 반복
MOV( G00 ) X0 Y0 Z0 ;
N1 INC( G91 ) LIN( G01 ) Z5 F500 ;
X100 ;
Y100 ;
X-100 ;
Y-100;
LOOP( M99 ) P1 L3 ;           - 메인 프로그램 반복 지정, 3회 반복
PEND ;

```

모션 프로그램 에디터를 이용하여 GX 시리즈에 저장된 모션 프로그램 파일을 Upload하여 편집 할 수 있습니다

2

모션 언어

2.1 모션 명령어

모션 명령어는 이송 시 이송 방법에 대한 형태를 정의하며 기계 구동 및 프로그램 운영등을 수행하는 명령입니다.

모션 명령어에 따라 원샷 지령과 모달 지령이 있습니다.

원샷 지령은 모션 명령어가 지령 된 블록에서만 기능이 유효하고 모달 지령은 같은 그룹의 모션 명령어가 지령 될 때까지 지령한 모션 명령어 코드가 다음 블록에서도 계속유효합니다.

구분	내용
원샷 지령	모션 명령어가 지령 된 블록에서만 유효
모달 지령	같은 그룹의 모션 명령어가 지령 될 때까지 유효

모션 명령어는 아래 모션 명령어 일람과 같이 정의되어 있고 여기에 정의되지 않은 모션 명령어가 지령되면 알람이 발생합니다.

동일 그룹의 모션 명령어가 같은 블록에 지령 될 때에는 나중에 지령 된 모션 명령어가 유효하고 서로 다른 그룹의 모션 명령어는 같은 블록에 여러 개 지령할 수 있습니다.

이벤트 코드	이벤트 내용
331	정의되지 않는 문자가 존재합니다.

1.  : 초기화 될 때 디폴트가 되는 ML를 파라미터에서 설정하는 ML
2.  : 초기화 될 때 디폴트가 되는 ML
3. ML = Motion Language

ML	G Code	Group	Define	M/B	Note
MOV	G00	1	급속이송	M	
LIN	G01		직선보간	M	
ACW	G02		원호보간	M	
ACC	G03		원호보간	M	
DWL	G04	0	휴지	B	
XYP	G17	16	XY 평면 선택	M	
PLN	G17.1		임의 평면 선택	B	
ZXP	G18		ZX 평면 선택	M	
YZP	G19		YZ 평면 선택	M	
INCH	G20	6	인치 단위 입력	M	
METR	G21		미터 단위 입력	M	
REF	G28	0	1 원점 이송	B	
REF2	G30		2 원점 이송	B	
REF3			3 원점 이송	B	
REF4			4 원점 이송	B	
SKP	G31	23	스킵기능 1	B	
SKP1	G31.1		스킵기능 1	B	
SKP2	G31.2		스킵기능 2	B	
MCS	G53	0	기계 좌표계 선택	B	
MCALL	G65	0	매크로 호출	B	
MMON	G66	12	매크로 모달 호출	B	
MMOF	G67		매크로 모달 호출 취소	B	
ABS	G90	3	절대모드 지령	M	
INC	G91		증분모드 지령	M	
WCS	G92	0	작업물 좌표계 설정	B	
SBON SBOF			Single Block On / Off 모드		
WAIT			신호 Wait		
PEND			프로그램 종료		

2.2 연산 명령

ML		Define	Note
조건문	IF [조건] GOTO Nxx	IF [#W1 LE 10] GOTO N1 ;	
	IF [조건] ELSE IEND	IF [#W1 LE 10] ; ELSE ; IEND ;	
	SWITCH[조건] CASE OO BREAK DEFAULT BREAK SWEND	SWITCH [#W1] ; CASE 0 ; A1 = 10 F1000 ; BREAK; DEFAULT ; A1 = 0 F1000 ; BREAK; SWEND ;	
반복문	WHILE [조건] DO Nxx WEND Nxx	WHILE [#W1 LE 10] DO 200 ; WEND 200 ;	
	WHILE [조건] WEND	WHILE [#W1 LE 10] ; WEND ;	
	FOR[INIT, 조건, STEP] FEND	FOR [#W1 = 1, #W1 LE 10, 1] ; FEND ;	
점프문	GOTO Nxx	GOTO N100 ;	

Command	Name	Format	Note
=	Substitute	#W1 = #W2 + #W3;	
->	Right Substitute	#W2 + #W3 -> #W1;	
+	Add	#W1 = #W1 + #W2 ;	
-	Subtract	#W1 = #W1 - #W2 ;	
*	Multiply	#W1 = #W1 * #W2 ;	
/	Divide	#W1 = #W1 / #W2 ;	
MOD	Reminder	IF[#W1 MOD 2 EQ 1] ;	
AND	And	IF[#W1 AND 2] ;	
OR	Or	IF[#W1 OR 2] ;	
XOR	Xor	IF[#W1 XOR 2] ;	
()	Parentheses	IF[(#W1 MOD 2) EQ 1] ;	[] 혼용 사용 가능

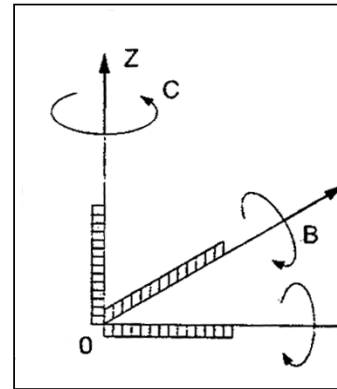
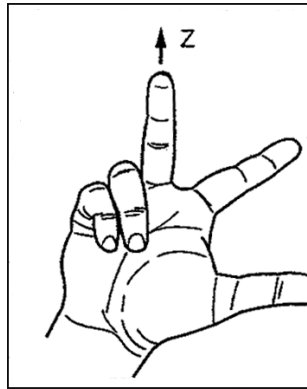
SIN	Sine	SIN(30) ;	
COS	Cosine	COS(30) ;	
TAN	Tangent	TAN(30) ;	
ATAN	Arc Tangent	ATAN(30) ;	
SQRT	Square Root	SQRT(30) ;	
ABSD	Absolute Data	ABSD(30) ;	
ROUND	Nearest Integer Value	ROUND(30) ;	
FIX		FIX(30) ;	
FUP		FUP(30) ;	
EQ	Match	IF[#W1 EQ #W2] ;	
<>	Mismatch	IF[#W1 <> #W2] ;	!= 혼용 사용 가능
>	Greater Than	IF[#W1 > #W2] ;	GT 혼용 사용 가능
<	Less Than	IF[#W1 < #W2] ;	LT 혼용 사용 가능
>=	Greater Than Or Equal to	IF[#W1 >= #W2] ;	GE 혼용 사용 가능
<=	Less Than Or Equal to	IF[#W1 <= #W2] ;	LE 혼용 사용 가능
BIN	BCD to BINARY	BIN(0X1234) ;	
BCD	BINARY to BCD	BCD(1234) ;	
SFTR	Right Shift	SFTR #B1.1 W10 N5 ;	W, N은 반드시 입력
		SFTR #B1.1 W10 N=5 ;	
		SFTR #B1.1 W=10 N5 ;	
		SFTR #B1.1 W=10 N=5 ;	
SFTL	Left Shift	SFTL #B1.1 W10 N5 ;	W, N은 반드시 입력
		SFTL #B1.1 W10 N=5 ;	
		SFTL #B1.1 W=10 N5 ;	
		SFTL #B1.1 W=10 N=5 ;	
MOVB	Move Bit	MOVB #B1.1 #B2.1 W5 ;	W 반드시 입력
		MOVB #B1.1 #B2.1 W=5 ;	
MOVD	Move Data	MOVD #B1 #B2 W5 ;	W 반드시 입력
		MOVD #B1 #B2 W=5 ;	
SETB	Set Bit	SETB #B1.1 W4 L1 D15 ;	W, L, N 반드시 입력
		SETB #B1.1 W=4 L=1 D=15 ;	
SETD	Set Data	SETD #B1 W4 D15 ;	W, D 반드시 입력
		SETD #B1 W=4 D=15 ;	

3

좌표계

3.1 좌표계

좌표계는 기계원점을 기준으로 하는 기계 좌표계와 프로그램의 원점을 기준으로 하는 작업물 좌표계가 있습니다. 좌표계는 주로 오른손 직교 좌표계를 사용하고 있습니다.



기계 좌표계

고유의 기계 원점을 가지고 있으며, 이 기계 원점을 기준으로 기계의 기계 좌표계를 정의합니다.

기계 좌표의 설정은 전원투입 후 수동 원점 이송을 실행하여 완료하면 됩니다.

제 2,3,4원점 설정 등의 기준이 되며 기계 원점에서 기계 좌표 값은 X0, Y0, Z0입니다.

작업물 좌표계

작업물 좌표계는 작업물의 이송 기준점이 원점으로 설정되는 좌표계입니다.

프로그램에서 WCS(G92) X_ Y_ Z_ 와 같이 작업물 좌표계를 설정 할 수 있습니다.

작업물 좌표계 원점은 절대지령의 기준점이 되고, 프로그램상의 원점을 기준으로 절대 좌표 [ABS(G90)] 값으로 X0 Y0 Z0 입니다.

3.2 기계 좌표계

[프로그램 입력 형식]

```
ABS MCS A1 = _ A2 = _ A3 = _ ;
G90 G53 X_ Y_ Z_ ;
```

[프로그램 입력 설명]

ABS(G90)	- 절대 지령 입력
MCS(G53)	- 기계 좌표계 선택 입력
X_ Y_ Z_	- 이송 위치 입력

기계 좌표계를 사용하기 위해서는 **MCS**을 지령합니다.

MCS은 원샷 **ML**이므로 지령한 블록에서만 유효합니다. 이 명령이 실행되면, **INC**의 증분 좌표계 지령도 일시적으로 무시됩니다.

이 **ML**은 최초 전원을 투입한 후 **REF**를 사용한 자동 기준점 복귀 또는 조작키를 이용한 수동원점복귀를 반드시 1회 이상 실행해야 유효합니다.

적용 예제

```
MCS( G53 ) ABS( G90 ) X-140 Y-120 ;    - X-140 Y-120 위치로 이동
WCS( G92 ) X0 Y0 ;                      - 작업물 좌표계를 변경하여 재 설정
REF2( G30 ) INC( G91 ) ;
MOV( G00 ) ABS( G90 ) X0 Y0 ;
PEND ;
```

3.3 작업물 좌표계 설정

[프로그램 입력 형식]

```
ABS WCS A1 = _ A2 = _ A3 = _ ;
G90 G92 X_ Y_ Z_ ;
```

[프로그램 입력 설명]

ABS(G90)	- 절대 지령 입력
WCS(G92)	- 작업물 좌표계 설정 입력
X_ Y_ Z_ / A1 = _ A2 = _ A3 = _	- 설정하고자 하는 작업물 좌표계의 현재 위치 입력

프로그램을 작성시 도면이나 작업물의 기준점을 설정하여 그 기준점으로부터 위치를 지령하면 프로그램을 간단하고 정확하게 작성할 수 있습니다. 그렇지만 작업물의 기준점이 어느 위치에 있는지 기계는 인식하지 못하므로 이 기준점을 **Gx**에 알려주어야 합니다. 이 때 절대지령으로 어떤 위치(제 1원점, 제 2원점)로 이동시키는 경우 프로그램원점으로부터의 좌표값을 지령하는 경우가 많은데, 이때 프로그램의 원점을 설정하는 **ML**입니다. 기준 위치를 잡기 위해서 원점이송 또는 제 2원점 이송을 수행한 후 **WCS**를 사용하여 작업물 좌표계를 설정하면 됩니다.

또한 자동 작업물 좌표계 설정 기능이 있습니다.

WCS를 이용하지 않고 파라미터 **P1900**에서 자동 작업물 좌표계 사용 여부와 파라미터 **P1908** 에서 **P1939** 까지 작업물 좌표계에 사용될 위치값을 입력합니다.

자동 작업물 좌표계 사용 여부를 설정하고 메인 프로그램을 수행하면 작업물 좌표계 위치 설정 파라미터에 입력한 값으로 작업물 좌표계가 설정됩니다.

자동 작업물 좌표계 설정 기능을 해제 하고자 하는 경우에는 자동 작업물 좌표계 사용 여부 파라미터를 **0**으로 설정하면 됩니다.

주
의

1. WCS 작업물 좌표계 설정을 사용할 경우에는 항상 원점이송을 먼저 실행한 후에 좌표계 설정을 하기 때문에 불 필요한 축 이송을 실행해야 하는 번거로움이 있습니다.
2. 프로그램 원점을 기점으로 한 좌표계를 작업물 좌표계라고 하며 일단 좌표계가 설정되면 이후에 지령되는 절대지령은 이 작업물 좌표계에서의 위치로 됩니다.

[관련 파라미터]

번호	내용
P1900	자동 Work 좌표계 사용 여부(0: X, 1: O)
P1908 – P1939	자동 Work 좌표계 위치 설정[PU]

적용 예제

```

REF( G28 ) INC( G91 ) X0 Y0 Z0 ;
WCS( G92 ) ABS( G90 ) X140 Y120 Z150 ;
REF2( G30 ) INC( G91 ) Z0 ;
MOV( G00 ) ABS( G90 ) X0 Y0 ;
PEND ;

```

3.4 평면 선택

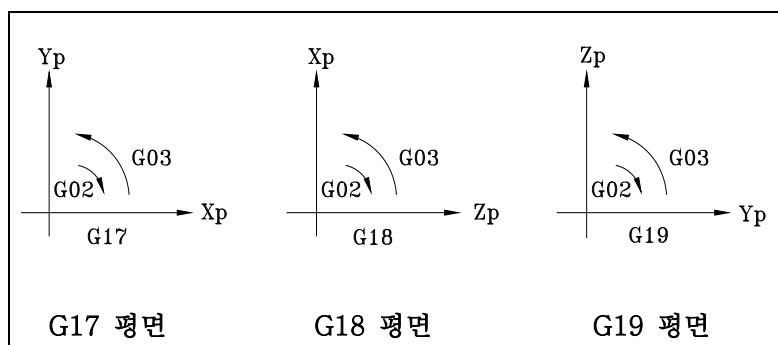
[프로그램 입력 형식]

```
[ XYP ] { ACW / ACC } A1 = _ A2 = _ { I_ J_ / R_ } F_ ;
[ ZXP ] { ACW / ACC } A1 = _ A3 = _ { I_ K_ / R_ } F_ ;
[ YZP ] { ACW / ACC } A2 = _ A3 = _ { J_ K_ / R_ } F_ ;
[ G17 ] { G02 / G03 } X_ Y_ { I_ J_ / R_ } F_ ;
[ G18 ] { G02 / G03 } X_ Z_ { I_ K_ / R_ } F_ ;
[ G19 ] { G02 / G03 } Y_ Z_ { J_ K_ / R_ } F_ ;
```

[프로그램 입력 설명]

XYP(G17)	- XY 평면 선택
ZXP(G18)	- ZX 평면 선택
YZP(G19)	- YZ 평면 선택
ACW(G02) / ACC(G03)	- 원호 보간 지령
X_ Y_ Z_	- 이송 위치
I_ J_ K_	- 원호의 중심 위치(벡터로 표시)
R_	- 반경
F_	- 이송 속도

원호 보간은 한 평면 내에서만 가능하기 때문에 **ML**를 이용한 평면 선택을 해야 합니다.



	적용 예제
--	-------

“ 원호 보간

MOV(G00) X0 Y0 Z0 ;

ABS(G90) XYP(G17) ACW(G02) X50 Y50 I50 F100 ;

ACC(G03) X0 Y0 R-50 ;

INC(G91) ACC(G03) X100. Y100. J100. ;

ACW(G02) X-100. Y-100. R-100. ;

ABS(G90) ZXP(G18) ACW(G02) Z50. X50. K50. F200. ;

ACC(G03) Z0. X0. R-50. ;

INC(G91) ACC(G03) Z100. X100. I100. ;

ACW(G02) Z-100. X-100. R-100. ;

ABS(G90) YZP(G19) ACW(G02) Y50 Z50 J50 F300 ;

ACC(G03) Y0 Z0 R-50 ;

INC(G91) ACC(G03) Y100 Z100 K100 ;

ACW(G02) Y-100 Z-100 R-100 ;

PEND ;

3.5 임의의 평면 선택

[프로그램 입력 형식]

```
PLN X Y / X A2 / A1 Y ;
G17.1 X Y / X A2 / A1 Y ;
```

[프로그램 입력 설명]

```
PLN( G17.1 )           - 임의 평면 선택
X Y | ZX | YZ          - XY 평면, ZX 평면, YZ 평면을 의미
X A2 | A1 Y            - X A2 평면, A1 Y 평면을 의미
```

임의의 두 축을 이용하여 평면을 선택하고자 할 때 사용합니다.

PLN X Y는 “XYP”과 같은 의미를 같습니다.

PLN을 이용하여 원호보간을 지령 할 때 필요한 평면 선택을 할 수 있습니다.

주의	PLN 지령할 때 평면을 나타내는 두 문자사이에 빈 공간을 입력해야 합니다. Ex] XY 평면을 지령하고자 할 때 <pre>PLN X Y ; - 바른 입력 PLN XY ; - 틀린 입력</pre>
----	---

이벤트 코드	이벤트 내용
384	원호 평면을 지정하지 않았습니다.
385	반지름을 지령하지 않았습니다.

	적용 예제
--	-------

“ 원호 보간

```
MOV(G00) X0 Y0 Z0 ;
ABS(G90) ;
PLN X Y ;           - XY 평면
ACW(G02) X50 Y50 I50 F100 ;
ACC(G03) X0 Y0 R-50 ;
INC(G91) ACC(G03) X100. Y100. J100. ;
ACW(G02) X-100. Y-100. R-100. ;
PEND ;
```

4

좌표치와 치수

4.1 절대 지령

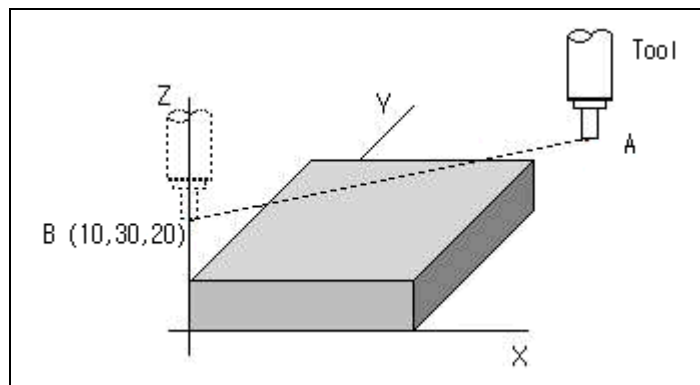
[프로그램 입력 형식]

```
ABS MOV A1 = _ A2 = _ A3 = _ ;  
G90 G00 X_ Y_ Z_ ;
```

[프로그램 입력 설명]

ABS(G90)	- 절대 지령 입력
MOV(G00)	- 위치 결정 입력
X_ Y_ Z_	- 이송 위치 입력

이동 종점의 위치를 절대좌표계의 위치(작업물 좌표계 원점을 기준으로 한 위치)로 지령하는 방식입니다.



적용 예제

ABS(G90) X10 Y30 Z20 ; - A 지점에서 B 지점으로 이동 시키는 명령

4.2 증분 지령

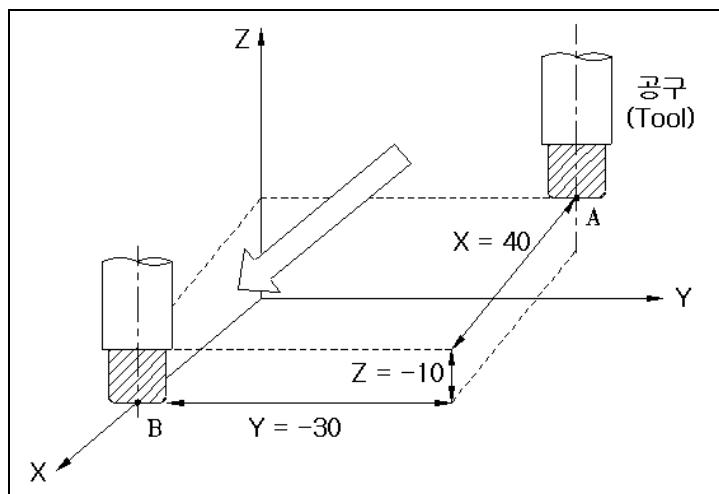
[프로그램 입력 형식]

```
INC MOV A1 = _ A2 = _ A3 = _ ;
G91 G00 X_ Y_ Z_ ;
```

[프로그램 입력 설명]

INC(G91)	- 증분 지령 입력
MOV(G00)	- 위치 결정 입력
X_ Y_ Z_	- 이송 위치 입력

현재 위치를 기준으로 다음 위치까지의 거리(이동량)를 지령하는 방식으로, 바로 전 위치를 기준으로 하여 현재 위치에 대한 좌표 값을 증분량으로 표시하며, 부호의 결정은 시점을 기준으로 종점이 어느 방향 인가에 따라 결정됩니다.



적용 예제

INC(G91) X40 Y-30 Z-10 ;

- A 점에서 B 점으로 이동 지령

4.3 Inch / Metric 단위 입력

[프로그램 입력 형식]

INCH / METR ;
G20 / G21 ;

[프로그램 입력 설명]

INCH(G20) - INCH 단위계 입력 모드
METR(G21) - MM 단위계 입력 모드

입력된 좌표값을 mm 또는 inch 단위로 지령하게 하는 MC로 도면 전체의 치수가 inch 또는 mm 좌표계로 입력되어 있을 때 기계의 이동단위를 inch나 Metric 시스템으로 변환하여 간단하게 프로그램을 작성할 수 있으며, INCH / METR이 입력된 블록을 포함한 하위 블록에 좌표값을 지배하는 모달 명령입니다.

[최소 설정 단위]

MC	G 코드	단위계	최소 설정단위
INCH	G20	Inch	0.0001 inch
METR	G21	Metric	0.001 mm

단위계의 변환은 프로그램의 선두에 좌표계를 설정하기 전에 단독 블록으로 지령해야 하며, 한 프로그램 안에서 inch 또는 Metric 을 변환하는 것은 좋지 않으므로 도면의 일부가 inch 또는 Metric 으로 되어 있는 경우는 환산하여 지령합니다.

[관련 파라미터]

번호	내용
P152	설정 단위계(0:Metric, 1:Inch)
P1850	지령 단위계(0:Metric, 1:Inch)

5

모션 기능

5.1 위치 결정

[프로그램 입력 형식]

```
[ ABS / INC ] MOV A1 = _ A2 = _ A3 = _ ;
[ G90 / G91 ] G00 X_ Y_ Z_ ;
```

[프로그램 입력 설명]

ABS(G90) / INC(G91)	- 절대 / 증분 지령
MOV (G00)	- 위치 결정 지령
X_ Y_ Z_ / A1 = _ A2 = _ A3 = _	- 이송 위치

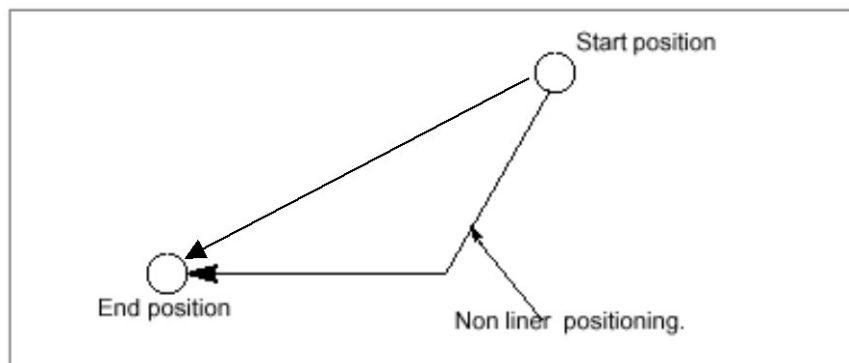
X,Y,Z 또는 축 번호로 지령 된 위치로 파라미터에 각 축별로 설정된 급속 이송 속도
로 이송하는 기능입니다.

이송 시 인포지션 검사가 완료된 후 다음 블록으로 진행합니다.

위치 결정 속도 **Override**로 이송 속도 변경이 가능합니다.

이송은 각 축별로 독립적으로 이송하며 거리가 짧은 축이 먼저 목표점에 도달하게
되어 직선을 이루지 못합니다.

또는 파라미터에서 보간형 위치 결정 사용 여부를 설정하여 직선형태로 이송되게
할 수 있습니다.



[관련 파라미터]

번호	내용
P2542 - P2573	위치 결정 지령의 Default 이송 속도
P2510 - P2541	위치 결정 지령의 최대 이송 속도

적용 예제

“ 위치 결정 ;

ABS(G90) MOV(G00) X0 Y0 Z0 ; - 절대 지령

X100 Y50 ;

Y100 ;

Z100 ;

INC(G91) X-100 ; - 증분 지령

Y-100 ;

Z-100 ;

PEND ;

5.2 직선 보간

[프로그램 입력 형식]

```
[ ABS / INC ] LIN A1 = _ A2 = _ A3 = _ F_ ;
[ G90 / G91 ] G01 X_ Y_ Z_ F_ ;
```

[프로그램 입력 설명]

ABS(G90) / INC(G91)	- 절대 / 증분 지령
LIN(G01)	- 직선 보간 지령 입력
X_ Y_ Z_ / A1 = A2 = _ A3 = _	- 이송 위치
F_	- 이송 속도

원하는 이송을 하기 위하여 지령 된 속도로 지령 된 위치까지 직선으로 각 축들을
동시 이송하는 기능입니다.

이송속도는 F_로 지령하고 이송 방법은 분당 이송을 사용합니다.

한번 지령 된 F_ 값은 모달로 다음 블록에서도 계속 유효합니다.

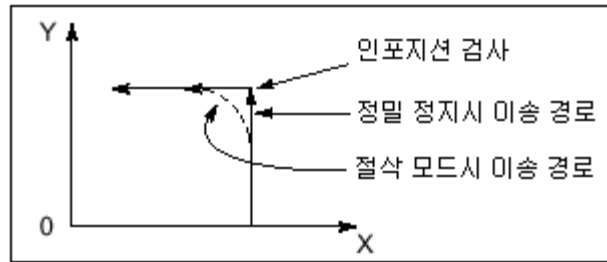
LIN 블록이나 이전 블록에서 F_ 코드에 의한 속도지정이 되어 있지 않으면 파라미
터에 설정된 **Default Feed**로 움직입니다.

또한 모든 물체는 관성을 가지고 있기 때문에 어떤 물체에 힘을 주어서 순간적으로
정지 시키거나 방향을 바꾸어 원하는 위치로 이동시키는 것은 어렵습니다.

일반적인 절삭모드에서는 이와 같은 문제를 보완하기 위해서 축의 이송을 가감속
할 때에 현재 블록을 미리 감속하고 다음 블록을 미리 가속하는 방법을 사용합니다.

```
MOV( G00 ) X50 Y0 ;
LIN( G01 ) Y50 F2000 ;
X0 ;
```

위의 이송 프로그램을 실행하면 Y축이 Y50의 좌표에 도달하기 전에 Y축을 미리 감
속하며, X축을 미리 가속합니다. 따라서, 코너 부분에서는 라운드 현상이 발생합니다.



정밀 정지 지령을 사용하면, 모터가 감속할 때에 파라미터에서 지정한 **In-Position** 범위 내에 축이 위치함을 **In-Position** 체크 후에 다음 축의 이송을 실행합니다.
따라서, 정밀 정지 기능을 사용하며 코너 부분에서 라운드 현상이 일어나는 것을 방지할 수 있습니다.

[관련 파라미터]

번호	내용
P2594 - P2601	보간 이송 지령의 Default 직선 이송 속도
P2602 - P2609	보간 이송 지령의 Default 회전 이송 속도
P2578 - P2585	보간 이송 지령의 최대 직선 이송 속도
P2586 - P2593	보간 이송 지령의 최대 회전 이송 속도

적용 예제

“ 직선 보간 ;

ABS(G90) X0 Y0 Z0 ;

ABS(G90) LIN(G01) X-50 F100 ; - 직선보간 지령

Y-40 ;

Z-30 ;

INC(G91) X50 Y40 Z30 ;

PEND ;

5.3 원호 보간

[프로그램 입력 형식]

```
[ XYP ] { ACW / ACC } A1 = _ A2 = _ { I_ J_ / R_ } F_ ;
[ ZXP ] { ACW / ACC } A1 = _ A3 = _ { I_ K_ / R_ } F_ ;
[ YZP ] { ACW / ACC } A2 = _ A3 = _ { J_ K_ / R_ } F_ ;
[ G17 ] { G02 / G03 } X_ Y_ { I_ J_ / R_ } F_ ;
[ G18 ] { G02 / G03 } X_ Z_ { I_ K_ / R_ } F_ ;
[ G19 ] { G02 / G03 } Y_ Z_ { J_ K_ / R_ } F_ ;
```

[프로그램 입력 설명]

XYP, ZXP, YZP	- 평면 지령
ACW(G02) / ACC(G03)	- 원호 보간 지령 입력
X_ Y_ Z_	- 이송 위치
I_ J_ K_	- 원호의 시점에서 중심까지의 벡터(방향 및 거리) R 지령 대신에 사용
R_	- 원호 반경
F_	- 이송 속도

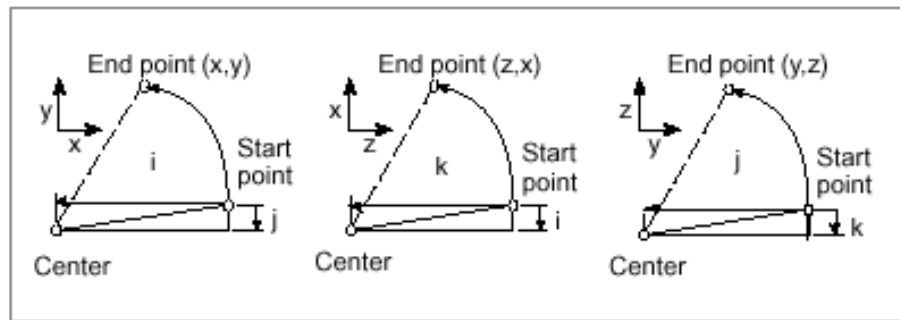
지령 된 속도로 지령 된 위치까지 지령 된 중심점을 기준으로 회전 이송하는 기능입니다. 여기서 속도는 접선 방향의 선 속도를 의미합니다.

작업물 좌표계 상의 X_ Y_ / X_ Z_ / Y_ Z_ 위치까지 I_ J_ / I_ K_ / J_ K_ 또는 R_ 에 의한 중심점을 기준으로 F_ 속도로 회전 이송합니다.

한번 지령 된 F_ 값은 모달로 다음 블록에서도 계속 유효합니다.

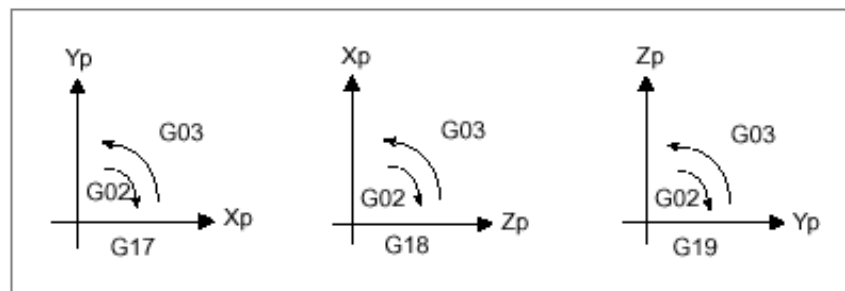
I_ J_ / I_ K_ / J_ K_ 에 의한 중심점 지령값은 절대 / 증분 지령에 상관 없이 시작위치에서 중심점까지의 증분 거리입니다. IO, JO, KO 지령인 경우는 각각 생략될 수 있습니다.

시작점과 끝점이 같은 경우에 I_ J_ K_ 지령에 의해서 360° 정원을 지령할 수 있습니다.



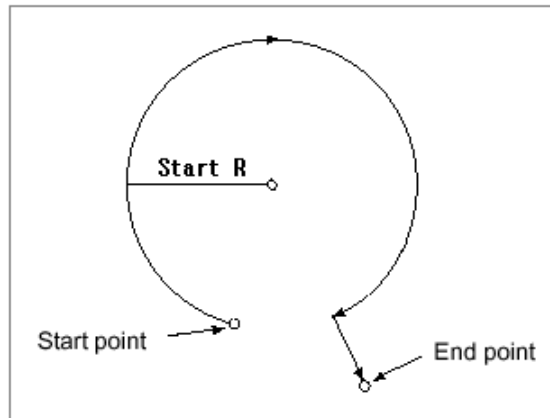
XYP은 XY평면, ZXP은 ZX평면, YZP은 YZ평면으로 정의됩니다.

각 평면에 대해서 중심점을 기준으로 ACW는 시계방향 회전이고 ACC는 반 시계 방향 회전에 이송합니다.



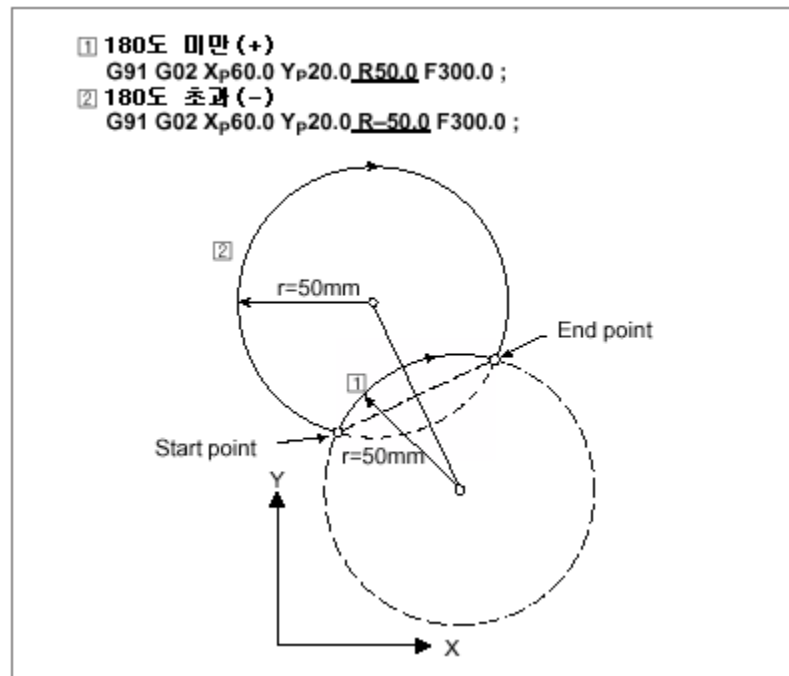
시작점에서 중심점까지 반경과 끝점에서 중심점까지의 반경에 대한 허용오차는 파라미터에서 정의할 수 있고 허용오차를 넘게 되면 알람이 발생합니다.

파라미터의 허용오차를 크게 설정하여 원호의 시작 반경과 종료 반경이 다른 경우에는 시작반경으로 회전하고 남은 이송은 직선으로 이송됩니다.



I_ J_ K_ 대신에 반경 R_에 의한 중심점 지령 시 중심점은 두 가지 형태가 될 수 있고 R값의 +, - 에 따라서 하나로 정해집니다. 시작점과 끝점 사이의 거리에 절반이 되는 거리와 R 지령값의 차이가 파라미터의 허용오차 보다 큰 경우에는 알람이 발생합니다.

I_ J_ K_와 R_이 동시에 지령 된 경우에 I_ J_ K_는 무시되고 R_ 의해서 중심점이 적용 됩니다. R_로는 360° 정원을 지령할 수 없습니다.



입력한 원호 반지름 이하의 원호에 대해서는 최고 절삭속도 이하로 제한되며, 원호 반경에 대한 절삭속도는 아래와 같이 계산됩니다.

$a = \text{최고 속도}^2 / \text{지령 원호 반경}$

$F = (a * \text{지령 원호 반경})^{1/2}$

이벤트 코드	이벤트 내용
364	원호의 중심점을 찾을 수 없습니다.
365	원호의 중심점 위치가 맞지 않습니다.

[관련 파라미터]

번호	내용
P4624	I, J, K 지령할 때 원호 중심점 재 생성 여부
P4557 - P4564	원호 허용 오차 설정
P4577	원호 이송속도 제한 기능 사용 여부
P4580 - P4587	원호 이송속도 제한 기능을 적용할 원호의 반경
P4588 - P4595	원호 이송속도 제한 기능의 최고 속도
P4596 - P4603	원호 이송속도 제한 기능의 최저 속도

적용 예제

“ 원호 보간

MOV(G00) X0 Y0 Z0 ;

ABS(G90) XYP(G17) ACW(G02) X50 Y50 I50 F100 ; - XY 평면

ACC(G03) X0 Y0 R-50 ;

INC(G91) ACC(G03) X100. Y100. J100. ;

ACW(G02) X-100. Y-100. R-100. ;

ABS(G90) ZXP(G18) ACW(G02) Z50. X50. K50. F200. ; - ZX 평면

ACC(G03) Z0. X0. R-50. ;

INC(G91) ACC(G03) Z100. X100. I100. ;

ACW(G02) Z-100. X-100. R-100. ;

ABS(G90) YZP(G19) ACW(G02) Y50 Z50 J50 F300 ; - YZ 평면

ACC(G03) Y0 Z0 R-50 ;

INC(G91) ACC(G03) Y100 Z100 K100 ;

ACW(G02) Y-100 Z-100 R-100 ;

XYP(G17) ACW(G02) I50 ;

- 정원

ACC(G03) J50 ;

ACW(G02) I50 J50 ;

ACW(G02) X-100 I30 R50 ;

- I 무시, R 적용

PEND ;

5.4 헬리컬 보간

[프로그램 입력 형식]

```
[ XYP ] { ACW / ACC } A1 = _ A2 = _ { I_ J_ / R_ } A3 = _ [ L_ ] F_ ;
[ ZXP ] { ACW / ACC } A1 = _ A3 = _ { I_ K_ / R_ } A2 = _ [ L_ ] F_ ;
[ YZP ] { ACW / ACC } A2 = _ A3 = _ { J_ K_ / R_ } A1 = _ [ L_ ] F_ ;
[ G17 ] { G02 / G03 } X_ Y_ { I_ J_ / R_ } Z_ [ L_ ] F_ ;
[ G18 ] { G02 / G03 } X_ Z_ { I_ K_ / R_ } Y_ [ L_ ] F_ ;
[ G19 ] { G02 / G03 } Y_ Z_ { J_ K_ / R_ } X_ [ L_ ] F_ ;
```

[프로그램 입력 설명]

XYP, ZXP, YZP - 평면 지령

ACW(G02) / ACC(G03) - 원호 보간 지령 입력

X_ Y_ Z_ - 이송 위치

I_ J_ K_ - 원호의 시점에서 중심까지의 벡터(방향 및 거리)

 R 지령 대신 사용

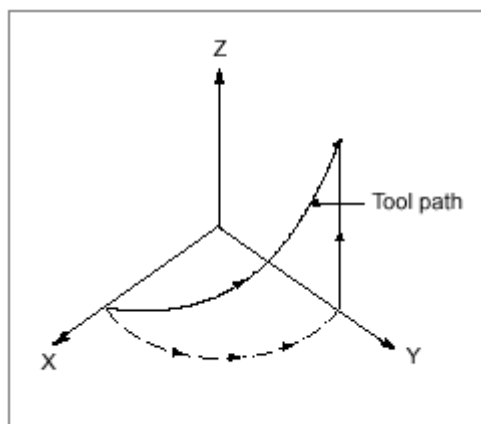
L_ - 다중 헬리컬 횟수

R_ - 원호 반경

F_ - 이송 속도

원호 보간 중에 원호 보간 평면상에 있지 않는 다른 한 축을 지령하여 원호 보간과 동기 시켜 직선 보간을 수행하는 기능입니다.

헬리컬 보간의 평면 선택	
XYP 평면	X, Y 축 원호보간, Z 축 직선보간
ZXP 평면	Z, X 축 원호보간, Y 축 직선보간
YZP 평면	Y, Z 축 원호보간, X 축 직선보간



한번 지령 된 F_ 값은 모달로 다음 블록에서도 계속 유효합니다.

적용 예제

“ 헬리컬 보간

MOV(G00) X0 Y0 Z0 ;

ABS(G90) XYP(G17) ACW(G02) X50 Y50 I50 Z50 F100 ; - XY 평면

ACC(G03) X0 Y0 R-50 ;

INC(G91) ACC(G03) X100 Y100 J100 Z100 ;

ACW(G02) X-100 Y-100 Z-100 R-100 ;

ABS(G90) ZXP(G18) ACW(G02) Z50 X50 K50 Y50 F200 ; - ZX 평면

ACC(G03) Z0 X0 R-50 ;

INC(G91) ACC(G03) Z100 X100 I100 Y100 ;

ACW(G02) Z-100 X-100 Y-100 R-100 ;

ABS(G90) YZP(G19) ACW(G02) Y50 Z50 J50 X50 F300 ; - YZ 평면

ACC(G03) Y0 Z0 R-50 ;

INC(G91) ACC(G03) Y100 Z100 X100 K100 ;

ACW(G02) Y-100 Z-100 R-100 ;

XYP(G17) ACW(G02) I50 Z50 ; - 정원

ACC(G03) J50 Z0 ;

ACW(G02) I100 J100 Z100 ;

PEND ;

5.5 스킵 기능

[프로그램 입력 형식]

```
{ SKP / SKP1 / SKP2 } A1 = _ A2 = _ A3 = _ F_ ;
{ G31 / G31.1 / G31.2 } X_ Y_ Z_ F_ ;
```

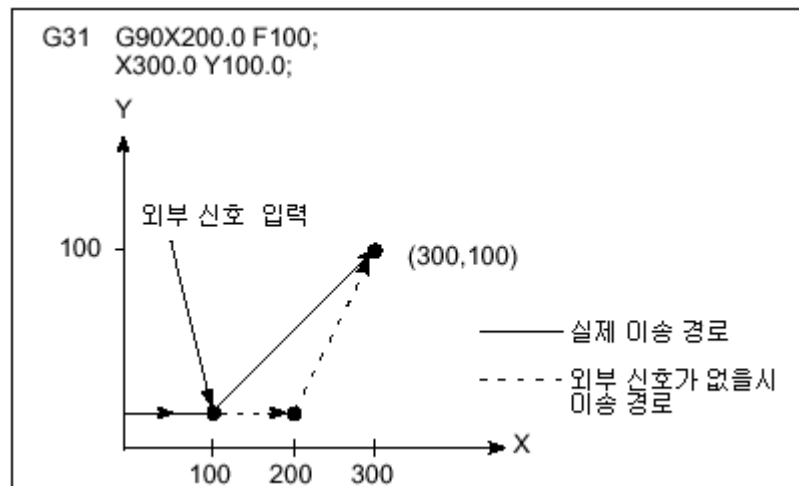
[프로그램 입력 설명]

SKP / SKP1 / SKP2	- 외부 신호별 지령 ML
X_ Y_ Z_	- 이송 위치
F_	- 이송 속도

지령 방법 및 축 이송 형태는 직선 보간과 동일하게 적용되면서 정의된 외부신호가 입력되면 이송이 중단되고 다음 블록으로 진행하게 되는 기능입니다.

이 때, 중단된 지점의 기계위치가 축 별로 **S796** 에서 **S827** 까지 입력됩니다.

이 기능은 작업물의 크기를 측정하거나 어떤 특정 위치를 이송 중에 알고자 할 때 사용됩니다.



적용 예제

```
ABS( G90 ) MPPM( G94 ) MOV( G00 ) X0 Y0 Z0 ;
```

```
SKP( G31 ) X100 F500 ;
```

- 외부 신호 1 입력 시 중단

```
SKP2( G31.2 ) Y100 ;
```

- 외부 신호 2 입력 시 중단

```
PEND ;
```

5.6 Teaching 기능

[프로그램 입력 형식]

```
MOV( G00 ) Px F1000 ;
```

[프로그램 입력 설명]

MOV(G00) - 위치 결정 지령
Px - P 레지스터에 저장된 위치정보

입력 형식에서의 **Px**는 P 레지스터에 설정한 축 정보의 값을 의미합니다.
만약, 1축과 3축을 선택하였다면 **Px**는 **A1 = 00 A3 = 00**와 같은 지령을 의미합니다.
지정할 수 있는 **Px**의 최대값은 파라미터 **P3824** 에서 **P3831** 에서 정의한 모션 데이터 테이블의 시작인덱스에 정의한 값보다 크게 되면 알람이 발생합니다.

이 벤트 코드	이벤트 내용
390	Teaching Point 지령 번호가 초과했습니다.

[관련 파라미터]

번호	내용
P4737 - P4744	Teaching 축 선택
P4749 - P4756	모션 데이터 테이블의 시작 인덱스
P20000 - P24999	모션 데이터 테이블

적용 예제

MOV(G00) P1 F1000 ; - 레지스터에 저장된 값 만큼 급속 위치 결정
P2 ; - P2의 값만큼 급속 위치 결정

파라미터 적용 예제	
Teaching 축 선택(1채널 - 1축,2축)	0000 0000 0000 0000 0000 0000 0000 0011
Teaching 축 선택(2채널 - 3축,4축)	0000 0000 0000 0000 0000 0000 0000 1100
모션 데이터 테이블 시작 인덱스(1채널)	2
모션 데이터 테이블 시작 인덱스(2채널)	7

모션 데이터 테이블 정보(PU = Default)				
인덱스	어드레스	값	Px	설명
0	20000			
1	20001			
2	20002	2		인덱스 2은 1채널에서 사용 가능한 Teaching Point 개수를 의미합니다. P0은 A1=10 A2=0, P1는 A1=0 A2=10와 같은 지령을 의미합니다. 만약, P2을 지령했다면 알람이 발생합니다.
3	20003	10000	P0	
4	20004	0		
5	20005	0	P1	
6	20006	10000		인덱스 7은 2채널에서 사용 가능한 Teaching Point 개수를 의미합니다. P0은 A3=10 A4=0, P1는 A3=0 A4=10, P2은 A3=10 A4=0와 같은 지령을 의미합니다. 만약, P3을 지령했다면 알람이 발생합니다.
7	20007	3		
8	20008	10000	P0	
9	20009	0		
10	20010	0	P1	
11	20011	10000		
12	20012	10000	P2	
13	20013	0		
14	20014	10000		

5.7 1 원점 이송

[프로그램 입력 형식]

```
[ ABS / INC ] REF X_ Y_ Z_ ;
[ G90 / G91 ] G28 X_ Y_ Z_ ;
```

[프로그램 입력 설명]

ABS(G90) / INC(G91) - 절대 / 증분 지령
 REF(G28) - 원점 이송 지령
 X_ Y_ Z_ - 원점 이송 해야 할 축에 대한 중간점 위치

작업물 좌표계상의 중간 점인 X_ Y_ Z_ 위치를 경유하여 지령한 축에 대하여 위치 결정 속도로 제 1원점인 기계원점으로 자동 이송하는 기능입니다.

주의	지령하지 않은 축은 원점으로 이송하지 않습니다.
----	----------------------------

[관련 파라미터]

번호	내용
P2510 - P2541	위치 결정 지령의 최대 이송 속도
P2542 - P2573	위치 결정 지령의 Default 이송 속도

적용 예제

“ 원점 이송

INC(G91) REF(G28) X0 Y0 Z0 ; - 현재 점을 중간 점으로 사용하며 원점 이송

ABS(G90) X10 Y20 Z30 ; - X10 Y20 Z30 위치로 이송

“ 원점 이송

INC(G91) REF(G28) Z0 ; - 현재 점을 중간 점으로 사용하여 Z축만 원점 이송

REF(G28) X0 Y0 ; - 현재 점을 중간 점으로 사용하여 X, Y축만 원점

이송

ABS(G90) X10 Y20 Z30 ;

5.8 2, 3, 4 원점 이송

[프로그램 입력 형식]

```
[ ABS / INC ] REF2 { P2 / P3 / P4 } X_ Y_ Z_ ;
[ G90 / G91 ] G30 { P2 / P3 / P4 } X_ Y_ Z_ ;
```

[프로그램 입력 설명]

ABS(G90) / INC(G91)	- 절대 / 증분 지령
REF2	- 2 원점 이송 지령
REF3 (G30)	- 3 원점 이송 지령
REF4	- 4 원점 이송 지령
P2 / P3 / P4	- 2, 3, 4 원점 종류 선택
X_ Y_ Z_	- 원점 이송 해야 할 축에 대한 중간점 위치

작업물 좌표계상의 중간 점인 X_ Y_ Z_ 위치를 경유하여 지령한 축에 대하여 위치 결정 속도로 파라미터에서 정의한 2, 3, 4 원점으로 자동 이송하는 기능입니다.

2원점은 P2로 지령하고 P2를 생략할 수도 있습니다. 3원점은 P3, 4원점은 P4로 지령하는 방법과 REF2, REF3, REF4의 ML를 이용하여 지정하는 방법이 있습니다.

주의	지령하지 않은 축은 원점으로 이송하지 않습니다.
----	----------------------------

[관련 파라미터]

번호	내용
P8202 - P8233	제 2 원점 위치
P8234 - P8265	제 3 원점 위치
P8266 - P8297	제 4 원점 위치

적용 예제

“ 2, 3, 4 원점 이송

ABS(G90) X0 Y0 Z0 ;	- X0 Y0 Z0 으로 이동
REF2(G30) P2 X30 Y50 Z10 ;	- X30 Y50 Z10 을 중간 점으로 하며 제 2 원점으로 이송
REF2(G30) P3 X0 ;	- X0 을 중간 점으로 하며 X축만 제 3 원점으로 이송
REF2(G30) P4 Y0 ;	- Y0 을 중간 점으로 하며 Y축만 제 4 원점으로 이송
PEND ;	

5.9 휴지

[프로그램 입력 형식]

```
DWL { X_ . P_ } ;
G04 { X_ / P_ } ;
```

[프로그램 입력 설명]

DWL(G04)	휴지 지령
X_	sec 단위 시간 지령
P_	msec 단위 시간 지령(1 / 1000 Sec)

지령한 시간 동안 프로그램의 축 이송 동작을 정지시키는 기능입니다.

지령한 X_ 또는 P_ 는 정지시간이 됩니다.

[관련 파라미터]

번호	내용
S828 - S835	Dwell Count

적용 예제

“ 휴지

MOV(G00) X0 Y0 Z0 ;

LIN(G01) X10 F500 ;

DWL(G04) X3 ;

- 시간 휴지

PEND ;

5.10 Single-Block

[프로그램 입력 형식]

```
SBON ;
SBOF ;
```

[프로그램 입력 설명]

```
SBON ;           - Single-Block 지령 설정
SBOF ;           - Single-Block 지령 해제
```

Single-Block Mode에서 프로그램을 실행 할 때 일반적으로 한 개의 블록만 수행합니다. 그러나 **SBON / SBOF** 명령어는 특정 블록의 연속된 동작을 수행하고자 할 때 사용합니다. 한 개의 블록만 수행되는 것이 아니라 **SBON** 부터 **SBOF**까지 수행됩니다.

SBON / SBOF는 모달 명령어이며 **SBON**이 **Default** 처리 됩니다.

SBON과 **SBOF**는 **ML** 단독으로 지령해야 합니다. **SBON** 이외의 다른 문자가 입력시에는 알람이 발생합니다.

이벤트 코드	이벤트 내용
331	정의되지 않은 문자가 존재합니다.

적용 예제

```
ABS( G90 ) LIN( G01 ) X100 Y100 Z100 ;
```

```
SBON ;           - Single Block 설정
```

```
X0 Y0 Z0 ;
```

```
X50 Y50 Z50 ;
```

```
SBOF ;           - Single Block 해제
```

5.11 EOM

[프로그램 입력 형식]

EOM ;

[프로그램 입력 설명]

EOM

- End of Motion

EOM은 EOM 지령 전 블록의 수행이 완료될 때까지 대기하는 ML입니다.

전 블록의 수행이 완료된 다음에는 EOM 블록 다음 블록부터 수행합니다.

특히, 매크로 변수의 값을 읽어 위치 지령 또는 보간 이송을 할 때 EOM을 사용하여 정확한 변수의 값을 읽어 들이기 위해 사용합니다.

적용 예제

```
#w1 = 100 ;
```

```
Eom ;
```

- #w1 = 100 수행이 완료될 때 까지 대기한다.

```
N1 A1=(#w1) F2000 ;
```

```
A1=-(#w1) F2000 ;
```

```
Eom ;
```

- A1=-(#w1) 수행이 완료될 때 까지 대기한다.

```
Goto N1 ;
```

```
Pend ;
```

5.12 신호 Wait

[프로그램 입력 형식]

```
WAIT( #B1.0 And #B1.1 ) ;
```

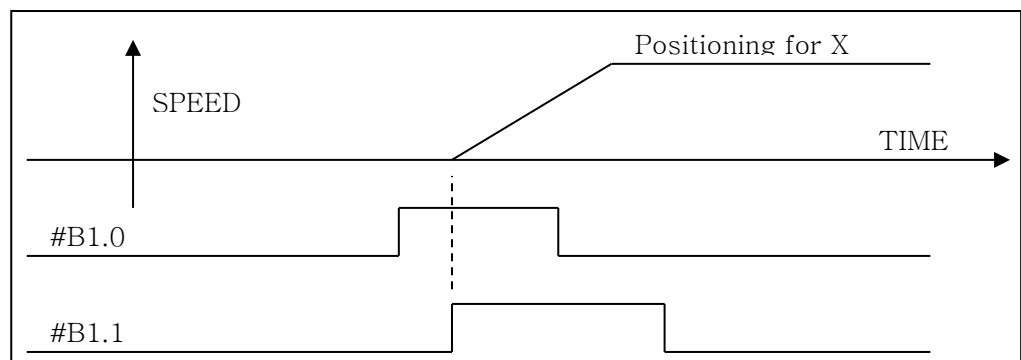
[프로그램 입력 설명]

WAIT	- I / O Wait
#B1.0 And #B1.1	- 조건식

WAIT는 조건식을 만족할 때까지 대기합니다.

조건이 만족하게 되면 현재 블록을 끝까지 수행하고 다음 블록으로 진행을 합니다.

단, 조건식에 사용할 수 있는 자료형은 **BIT**형만 가능합니다.



적용 예제

```
WAIT(#B1.0AND#B2.0EQ 1) ;
```

- #B1.0과 #B2.0의 AND한 결과값이 1과 같을 때
까지 대기

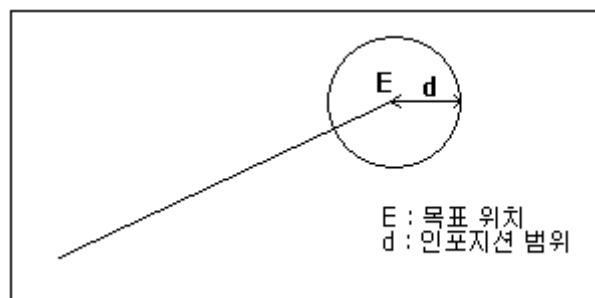
6

이송 기능

6.1 위치결정의 이송속도 지령

위치 결정은 위치 결정 지령 시 공구가 움직이는 형태입니다. 위치 결정은 지령 블록의 위치 이동이 완료되고 인포지션 검사를 수행하고 나서 다음 블록으로 진행 합니다. 위치 결정속도는 파라미터에 축별로 정의되어 있습니다.

인포지션이란 도달한 위치가 목표위치로부터 허용범위 이내에 위치해 있는 것을 말 하며 범위는 파라미터에서 축 별로 정의 되어집니다.



오버라이드 파라미터 테이블에 0으로 설정되어 있으면 이동은 수행되지 않습니다.

F_ 지령으로 위치 결정 이송 속도를 지령할 수 있습니다.

여기서 지령되는 F_ 지령은 원샷 지령입니다.

[관련 파라미터]

번호	내용
P2836	Mp 모드에서 Feed Override 적용 여부(0: X, 1: O)
P2837	Mc 모드에서 Feed Override 적용 여부(0: X, 1: O)
P2510 - P2541	위치 결정 지령의 최대 이송 속도
P2542 - P2573	위치 결정 지령의 Default 이송 속도

[적용 방법]

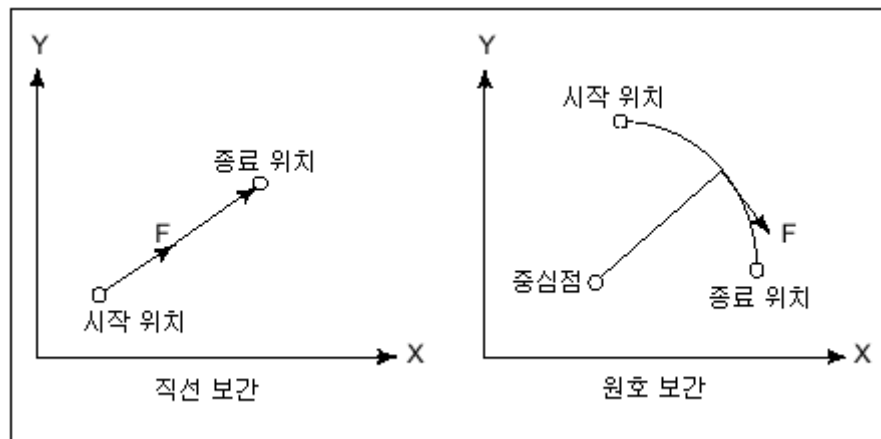
번호	적용 방법	내용
1	F1000	현재 설정된 축에 대해서 Feed 적용
2	F = 1000	현재 설정된 축에 대해서 Feed 적용
3	F = (#w1)	변수를 활용하여 Feed 적용
4	F1 = 1000 F2= 1500 F3 = 2000	각 축마다 Feed를 설정할 수 있다.

5	F1 = (#w1) F2 = (#w2) F3 = (#w3)	각 축에 변수를 활용하여 Feed 적용
---	---	-----------------------

6.2 보간 이송의 속도 지령

보간 이송은 직선 보간, 원호 보간 등의 지령 시 **F** 지령의 속도에 의해 이송되는 형태입니다. 이송 방법은 분당 이송(MMPM)이 있습니다.

보간 이송은 접선 방향 속도가 지령 속도가 되도록 유지하면서 제어를 수행합니다.



오버라이드 파라미터 테이블에 0으로 설정되어 있으면 이송은 수행되지 않습니다.

한번도 **F**가 지령 되지 않거나 **F0**으로 지령 되면 파라미터에 정의되어 있는 이송속도 **Default** 값으로 이송하고 파라미터에 정의된 이송속도 상한값을 넘는 이송이 지령 되면 알람이 발생합니다.

[관련 파라미터]

번호	내용
P2836	Mp 모드에서 Feed Override 적용 여부(0: X, 1: O)
P2837	Mc 모드에서 Feed Override 적용 여부(0: X, 1: O)
P2578 - P2585	보간 이송 지령의 직선 최대 이송 속도
P2586 - P2593	보간 이송 지령의 회전 최대 이송 속도
P2594 - P2601	보간 이송 지령의 Default 직선 이송 속도
P2602 - P2609	보간 이송 지령의 Default 회전 이송 속도

[적용 방법]

번호	적용 방법	내용
1	F1000	현재 설정된 축에 대해서 Feed 적용
2	F = 1000	현재 설정된 축에 대해서 Feed 적용
3	F = (#w1)	변수를 활용하여 Feed 적용

7

제어 및 연산 명령

[Type 2]

```

IF[ #W1 GT 10 ] ;           3중 복수지령 가능
조건이 참일 때 처리 Block ;   - 조건이 성립하는 경우
ELSE ;
조건이 거짓일 때 처리 Block ; - 조건이 성립하지 않는 경우
IEND ;

```

[Type 2 예제]

```

IF[ #W1 GT 10 ] ;
  IF[ #W2 GT 20 ] ;
    #W1 = 0 ;               - 조건이 성립하는 경우
  ELSE ;
    #W1 = 100 ;             - 조건이 성립하지 않는 경우
  IEND ;
ELSE ;
  IF[ #W3 GT 50 ] Goto N1 ; - 조건이 성립하지 않는 경우
IEND ;

```

조건식은 다음과 같이 비교하는 2개의 변수 또는 변수와 정수의 사이에 비교 연산자를 쓰고 전체를 괄호 [] 또는 ()로 묶습니다. 변수가 변하도록 식을 쓸 수도 있습니다.

연산자는 다음과 같이 2문자의 알파벳으로 구성되고, 크거나, 작거나, 또는 같은지를 판별합니다. 부등호는 사용할 수 없으므로 주의하여 주십시오.

연산자	의미
EQ	같다
NE	같지 않다(!=)
GT	보다 크다(>)
GE	같거나 크다(>=)
LT	보다 작다(<)
LE	같거나 작다(<=)

조건 분기(SWITCH 문)

SWITCH 문에 이어 조건식을 기술합니다.

조건식이 성립하면, CASE OO 으로 분기하고, 성립하지 않을 때는 DEFAULT 지령이 있는 경우에는 DEFAULT Block으로 이동하여 Block을 실행하며, DEFAULT Block이 없는 경우에는 SWEND Block으로 이동하여 Block을 실행합니다.

```

SWITCH[ 조건식 ] ;
CASE OO ;
    조건이 OO과 같을 때 실행되는 Block ;           조건이 성립하는 경우
    BREAK ;
DEFAULT ;
    조건이 OO과 같지 않을 때 실행되는 Block ;       조건이 성립하지 않는 경우
    BREAK ;
SWEND ;
  
```

조건식의 입력형태는 다음과 같다.

```

SWITCH[ #W1 ] ;
CASE 0 ;
    A1 = 10 F1000 ;                               #W1의 결과값이 0이면 실행
    BREAK ;
CASE 1 ;
    A1 = 20 F1000 ;                               #W1의 결과값이 1이면 실행
    BREAK ;
DEFAULT ;
    A1 = -10 F1000 ;                              #W1의 결과값이 0 또는 1이 아닌 경우 실행
    BREAK ;
SWEND ;
  
```

CASE OO와 BREAK, DEFAULT는 SWITCH ~ SWEND에 존재해야 합니다.

또한 CASE OO은 개수와 상관없이 사용할 수 있습니다.

만약 CASE OO를 다중으로 사용했다면 맨 처음 지령된 CASE OO을 처리합니다.

DEFAULT는 SWITCH ~ SWEND 에서 CASE OO 문 위에 지령할 수 있으며 또한 위의 조건식 입력형태처럼 지령할 수 있습니다.

만약 CASE OO 또는 DEFAUT에서 BREAK를 지령하지 않았다면 다음으로 지령된 BREAK를 처리하고 SWEND로 이동됩니다.

```

1  SWITCH[ #W1 ] ;
2    CASE 0 ;
3      A1 = 10 F1000 ;
4    CASE 1 ;
5      A1 = 20 F1000 ;
6    BREAK ;
7  DEFAULT ;
8    A1 = -10 F1000 ;
9    BREAK ;
10 SWEND ;

```

만약, #W1의 값이 0이라면 실행순서는 다음과 같다.
1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 10 의 순서로 실행된다.

반복(WHILE 문)

WHILE 문에 이어 조건식을 기술합니다.

조건식이 성립하고 있는 동안 DO부터 WEND간의 프로그램을 실행합니다.

조건이 성립하지 않을 때는 WEND의 다음 Block으로 진행합니다.

[Type 1]

```

WHILE[ 조건식 ] DO m ;          - m = 1, ... 10( 10중까지 가능 )
조건이 참일 때 실행되는 Block ; - 조건이 성립하는 경우( 반복해서 실행됨 )
WEND m ;
:                               - 조건이 성립하지 않는 경우

```

[Type 2]

```

WHILE[ 조건식 ] ;
조건이 참일 때 실행되는 Block ; - 조건이 성립하는 경우
WEND ;
:                               - 조건이 성립하지 않는 경우

```

조건식이 성립하고 있는 동안 WHILE에 이은 DO부터 WEND 사이를 실행합니다.

조건식이 성립하지 않을 때는 DO에 대응하는 WEND의 다음으로 진행합니다.

조건식과 연산자는 IF문과 같습니다.

DO와 WEND에 이은 수치는 실행하는 범위를 지정하는 식별 번호로 1, 2, ~ 10을 사용할 수 있습니다.

DO ~ WEND에서 사용하는 식별번호(1 ~ 10)는 몇 번 사용하셔도 괜찮습니다.

그러나 반복 Loop가 엇갈리는 프로그램은 알람이 발생합니다.

조건식의 입력형태는 다음과 같다.

```
FOR[ #W1 = 1; #W1 <= 10; 1 ] ;
```

```
#W1 = 1
```

- 초기값

```
#W1 <= 10
```

- 조건식에서의 조건을 나타내는 부분

```
1
```

- 증분값

#W1의 값에 초기값으로 1이 입력되고 #W1 <= 10의 조건에 만족하면 증분 값인 1을 증가시키고 FEND 까지 수행한 다음 다시 #W1 <= 10의 조건을 비교한다. 계속 수행을 하다가 #W1 = 11 이 됐을 때 #W1 <= 10의 조건이 성립하지 않으므로 FEND 다음 Block를 수행 하게 됩니다.

조건식이 성립하고 있는 동안 FOR부터 FEND 사이를 실행합니다.

조건식이 성립하지 않을 때는 FEND 의 다음으로 진행합니다.

조건식과 연산자는 IF 문과 같습니다.

FOR ... FEND 의 다중화는 10 중까지 가능합니다.

- 주의 -

1. DO m을 앞에 WEND m을 뒤에 지령하여야 합니다.

```
WEND 1 ;
```

```
:
```

```
DO 1 ;
```

- 불가

2. DO m과 WEND m과는 동일 프로그램내에서 1 대 1로 대응해야 합니다.

```
DO 1 ;
```

```
:
```

```
DO 1 ;
```

```
:
```

```
WEND 1 ;
```

- 불가

```
DO 1 ;
```

```
:
```

```
WEND 1 ;
```

```
:
```

```
WEND 1 ;
```

- 불가

3. 같은 식별 번호를 몇 번이라도 사용할 수 있습니다.

```
DO 1 ;
```

```
:
```

```
END 1 ;
```

- ```

:
DO 1 ;
:
END 1 ;

```
4. DO의 다중도는 10중까지입니다.
- ```

DO 1 ;
:
DO 2 ;
:
DO 10 ;
:
WEND 10 ;
:
WEND 2 ;
:
WEND 1 ;

```
5. DO의 범위는 교대로 됩니다.
- ```

DO 1 ;
:
DO 2 ;
:
WEND 1 ;
:
WEND 2 ;

```
6. DO의 범위에서 밖으로 분기할 수 있습니다.
- ```

DO 1 ;
:
GOTO 9000 ;
:
WEND 1 ;
:
N9000 ;

```
7. DO의 범위 밖에서 안으로 분기할 수는 없습니다.
- ```

GOTO 9000 ;
:
DO 1 ;

```

```

:
N9000 ;
:
WEND 1 ; - 불가
DO 1 ;
:
N9000 ;
:
WEND 1 ;
:
GOTO 9000 ; - 불가

```

8. DO의 범위에서 커스텀 매크로 본체 또는 부 프로그램을 호출할 수 있습니다. DO의 다중도는 커스텀 매크로 본체 또는 부 프로그램 내에서도 10중까지 가능합니다.

```

DO 1 ;
:
MCALL(G65) ... ; - 가능
:
MMON(G66) ... ; - 가능
:
MMOF(G67) ; - 가능
:
WEND 1 ;
:
DO 1 ;
:
LOOP(M98) ... ; - 가능
:
WEND 1 ;

```

## 7.2 연산 명령

변수의 정수, 치환

- #Fi = #Fj ;

가법형 연산

- #Fi = #Fj + #Fk ;  
 - #Fi = #Fj - #Fk ;  
 - #Fi = #Fj OR #Fk ;  
 - #Fi = #Fj XOR #Fk ;

승법형 연산

- #Fi = #Fj \* #Fk ;  
 - #Fi = #Fj / #Fk ;  
 - #Fi = #Fj AND #Fk ;

연산의 조합

연산의 우선순은 변수, 승법형 연산, 가감형 연산 순으로 됩니다.

#Wi = #Wj + #Wk \* SIN[ #W1 ] ;

또한 연산 순서를 우선하고 싶은 부분을 [ ] 또는 ( ) 로 묶을 수 있습니다.

[ ] 또는 ( )는 변수의 [ ] 또는 ( )를 포함하여 10중까지 가능합니다.

#Wi = SIN[ [ [ #Wj + #Wk ] \* #Wl + #Wm ] \* #Wn ] ;

수학 함수

- #Fi = SIN[ #Fj ] ;  
 - #Fi = COS[ #Fj ] ;  
 - #Fi = TAN[ #Fj ] ;  
 - #Fi = ATAN[ #Fj ] ;  
 - #Fi = SQRT[ #Fj ] ;

- #Fi = ABS[ #Fj ] ;  
 - #Fi = ROUND[ #Fj ] ;  
 - #Fi = AND[ #Fj ] ;  
 - #Fi = OR[ #Fj ] ;  
 - #Fi = FIX[ #Fj ] ;  
 - #Fi = FUP[ #Fj ] ;

## Round 사용법

1. 연산 지령 혹은 IF, WHILE 의 조건식에 사용될 때는 통상의 소수점에 대해 반올림을 합니다.

|                                    |                                        |
|------------------------------------|----------------------------------------|
| #W1 = ROUND[ 1.2345 ] ;            | #W1 = 1.0 이 됩니다.                       |
| IF[ #W1 LE ROUND[ #W2 ] ] GOTO 10; | #W2 가 3.567 일 때 ROUND[ #2 ]는 4.0 이 됩니다 |

2. 어드레스에 대한 지령 중에서 사용될 때는 그 어드레스의 최소 설정 단위로 반올림 됩니다.

|                              |                                                                 |
|------------------------------|-----------------------------------------------------------------|
| LIN(G01) X[ ROUND[ #W1 ] ] ; | #W1 이 1.4567 에서 X의 최소 설정 단위가 0.001 일 때 이 블록은 G01 X1.456; 로 됩니다. |
|------------------------------|-----------------------------------------------------------------|

### 적용 예제

|                                 |                                             |
|---------------------------------|---------------------------------------------|
| N1 #W1 = 1.2345 ;               |                                             |
| N2 #W2 = 2.3456;                |                                             |
| N3 INC(G91) LIN(G01) X#W1 F100; | - X1.235로 동작함.                              |
| N4 X#W2;                        | - X2.346로 동작함.                              |
| N5 X-[#W1+#W2];                 | - 1.2345 + 2.3456 = 3.5801 이므로 X3.580로 동작함. |
| N6 X-[ROUND[#W1]+ ROUND[#W2]];  | - 1 + 2 = 3 이므로 X3.로 동작                     |

## BCD to BINARY

### [ 프로그램 입력 형식 ]

```
BIN() ;
```

### [ 프로그램 입력 설명 ]

BIN                                    - BCD to BINARY 명령어  
( )                                    - 변경하고자 하는 값

BIN 명령어는 BCD data를 Binary data로 변경하는 명령어입니다.  
오직 정수형 또는 상수의 자료만 입력 및 변경 가능 합니다.

### [ 입력 인자 자료형 ]

|     | 비트( B ) | 정수( W ) | 실수( F ) | 상수 |
|-----|---------|---------|---------|----|
| 자료형 | X       | O       | X       | O  |

### 적용 예제

BIN( 0X1234 ) ;                    - 0X1234의 값을 Binary Data로 변경. ( 단, 0X는 대문자로 )

## BINARY to BCD

### [ 프로그램 입력 형식 ]

```
BCD() ;
```

### [ 프로그램 입력 설명 ]

BCD                                    - BINARY to BCD  
( )                                    - 변경하고자 하는 값

BCD 명령어는 Binary data를 BCD data로 변경하는 명령어입니다.  
오직 정수형 또는 상수의 자료만 입력 및 변경이 가능 합니다.

### [ 입력 인자 자료형 ]

|     | 비트( B ) | 정수( W ) | 실수( F ) | 상수 |
|-----|---------|---------|---------|----|
| 자료형 | X       | O       | X       | O  |

|       |
|-------|
| 적용 예제 |
|-------|

BCD( #W1 ) ;                      - #W1에 저장된 값을 BCD Data로 변경.

BCD( 1234 ) ;                    - 1234의 값을 BCD Data로 변경.

## Bit Right Shift

### [ 프로그램 입력 형식 ]

```
SFTR #B1.1 W10 N5 ;
SFTR #B1.1 W10 N = 5 ;
SFTR #B1.1 W = 10 N5 ;
SFTR #B1.1 W = 10 N = 5 ;
```

### [ 프로그램 입력 설명 ]

SFTR                      - Bit Right Shift 지령 입력  
 #B1.1                    - Leading Bit Number 입력  
 W10                      - Bit Width  
 N5                        - Number to be Shifted

특정한 **Bit Number** 와 **Bit Width** 만큼 오른쪽으로 이동시켜 **Bit String**를 재 지정합니다. 입력방식은 위의 형태로 입력 가능합니다. 그러나 **N, W** 는 반드시 입력해야 합니다. 또는 **N** 의 값과 **W** 의 값을 반드시 입력해야 합니다.

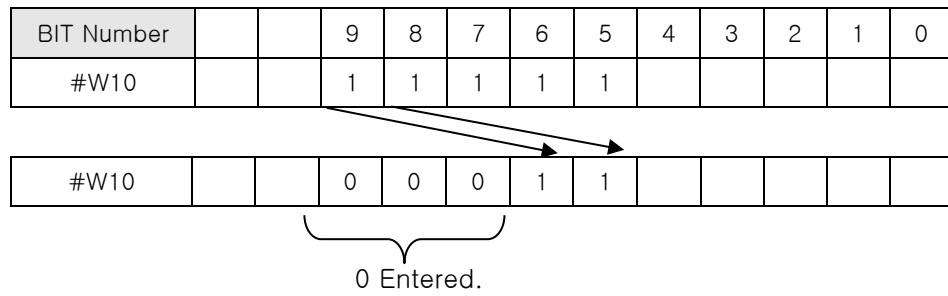
위의 입력방식을 제외한 다른 입력방식은 지원하지 않습니다.

입력 인자 자료형이 맞지 않을 때는 알람이 발생합니다.

### [ 입력 인자 자료형 ]

|          | 비트( B ) | 정수( W ) | 실수( F ) | 상수 |
|----------|---------|---------|---------|----|
| 시작 비트 숫자 | O       | X       | X       | X  |
| Bit 너비   | X       | O       | X       | X  |
| 이동량      | X       | O       | X       | X  |

SFR #B10.0A N3 W5;



SFTR 명령어에서 Bit Width보다 SHIFT의 수가 크면 모든 데이터는 특정한 Bit Width에서 0으로 설정됩니다.

| 이<br>벤트<br>코드 | 이벤트 내용                  |
|---------------|-------------------------|
| 378           | SHIFT에 사용할 수 없는 자료형입니다. |

|       |
|-------|
| 적용 예제 |
|-------|

SFTR #B0000001 W10 N5 ;

SFTR #B0.01 W10 N5 ;

SFTR #B.01 W10 N5 ;

- 3개의 예제는 다 같은 동작을 한다.  
다 같은 표현식입니다.

## Bit Left Shift

### [ 프로그램 입력 형식 ]

SFTL #B1.1 W10 N5 ;  
 SFTL #B1.1 W10 N = 5 ;  
 SFTL #B1.1 W = 10 N5 ;  
 SFTL #B1.1 W = 10 N = 5 ;

### [ 프로그램 입력 설명 ]

SFTL                      - Bit Left Shift 지령 입력  
 #B1.1                  - Leading Bit Number 입력  
 W10                     - Bit Width



N5

- Number to be Shifted

특정한 **Bit Number** 와 **Bit Width** 만큼 왼쪽으로 **Shift**하여 **Bit String**를 재 지정합니다. 입력방식은 위의 형태로 입력 가능합니다. 그러나 **N, W** 는 반드시 입력해야 합니다. 또는 **N** 의 값과 **W** 의 값을 반드시 입력해야 합니다.

위의 입력방식을 제외한 다른 입력방식은 지원하지 않습니다.

입력 인자 자료형이 맞지 않을 때는 알람이 발생합니다.

## [ 입력 인자 자료형 ]

|          | 비트 ( B ) | 정수 ( W ) | 실수 ( F ) | 상수 |
|----------|----------|----------|----------|----|
| 시작 비트 숫자 | O        | X        | X        | X  |
| Bit 너비   | X        | O        | X        | X  |
| 이동량      | X        | O        | X        | X  |

|                      |   |   |   |   |   |   |  |  |   |   |   |   |
|----------------------|---|---|---|---|---|---|--|--|---|---|---|---|
| SFTL #B10.0A N5 W10; |   |   |   |   |   |   |  |  |   |   |   |   |
| BIT Number           | F | E | D | C | B | A |  |  |   |   |   |   |
| #W10                 | 1 | 1 | 0 | 0 | 0 | 1 |  |  |   |   |   |   |
| BIT Number           |   |   |   |   |   |   |  |  | 3 | 2 | 1 | 0 |
| #W11                 |   |   |   |   |   |   |  |  | 0 | 1 | 0 | 1 |

0 Entered.

|            |   |   |   |   |   |   |  |  |   |   |   |   |
|------------|---|---|---|---|---|---|--|--|---|---|---|---|
| BIT Number | F | E | D | C | B | A |  |  |   |   |   |   |
| #W10       | 1 | 0 | 0 | 0 | 0 | 0 |  |  |   |   |   |   |
| BIT Number |   |   |   |   |   |   |  |  | 3 | 2 | 1 | 0 |
| #W11       |   |   |   |   |   |   |  |  | 1 | 0 | 0 | 0 |

|      |  |  |  |   |   |   |   |   |  |  |  |  |
|------|--|--|--|---|---|---|---|---|--|--|--|--|
| #W10 |  |  |  | 0 | 0 | 0 | 1 | 1 |  |  |  |  |
|------|--|--|--|---|---|---|---|---|--|--|--|--|

0 Entered.

SFTL 명령어에서 Bit Width보다 SHIFT의 수가 크면 모든 데이터는 특정한 Bit Width에서 0으로 설정됩니다.

| 이<br>벤트<br>코드 | 이벤트 내용                  |
|---------------|-------------------------|
| 372           | SHIFT에 사용할 수 없는 자료형입니다. |

|       |
|-------|
| 적용 예제 |
|-------|

SFTL #B0000001 W10 N5 ;

SFTL #B0.01 W10 N5 ;      - 3개의 예제는 다 같은 동작을 수행합니다.

SFTL #B.01 W10 N5 ;

## Move Bit( MOVB )

### [ 프로그램 입력 형식 ]

MOVB #W1.00 DW1.00 W10 ;

### [ 프로그램 입력 설명 ]

MOVB                      - Move Bit 지령 입력  
 #W1.00                  - 복사할 원본이 되는 레지스터 입력  
 DW1.00                  - 복사할 목표가 되는 레지스터 입력  
 W10                      - 복사할 폭 입력

임의의 비트 값 한 개를 원하는 위치로 복사합니다.

또는 어떤 범위의 비트 값들을 원하는 위치로 복사합니다. 원본이 되는 비트와 목표가 되는 비트를 지정하고 폭을 지정하면, 원본에서 목표로 지정한 범위에 해당하는 비트 값들이 복사됩니다.

### [ 입력 인자 자료형 ]

|          | 비트( B ) | 정수( W ) | 실수( F ) | 상수 |
|----------|---------|---------|---------|----|
| 복사할 레지스터 | O       | X       | X       | X  |
| 복사될 레지스터 | O       | X       | X       | X  |
| 이동될 값    | X       | O       | X       | X  |

| 이<br>벤트<br>코드 | 이벤트 내용                 |
|---------------|------------------------|
| 374           | MOVB에 사용할 수 없는 자료형입니다. |

|       |
|-------|
| 적용 예제 |
|-------|

MOVB #W1.00 #W10.00 W5 ; - #W1.00에서 비트 5개를 #W10.00부터 5개를 복사

## Move Data( MOVD )

## [ 프로그램 입력 형식 ]

|                    |
|--------------------|
| MOVD #W1 DW1 W10 ; |
|--------------------|

## [ 프로그램 입력 설명 ]

|      |                      |
|------|----------------------|
| MOVD | - Move Data 지령 입력    |
| #W1  | - 복사할 원본이 되는 레지스터 입력 |
| DW1  | - 복사할 목표가 되는 레지스터 입력 |
| W10  | - 복사할 폭 입력           |

임의의 레지스터 값 한 개를 원하는 위치로 복사합니다.

또는 어떤 범위의 레지스터 값들을 원하는 위치로 복사합니다.

원본이 되는 레지스터와 목표가 되는 레지스터를 지정하고 폭을 지정하면, 원본에서 목표로 지정한 범위에 해당하는 레지스터 값들이 복사됩니다.

## [ 입력 인자 자료형 ]

|          | 비트( B ) | 정수( W ) | 실수( F ) | 상수 |
|----------|---------|---------|---------|----|
| 복사할 레지스터 | X       | O       | X       | X  |
| 복사될 레지스터 | X       | O       | X       | X  |
| 이동될 값    | X       | O       | X       | X  |

| 이벤트 코드 | 이벤트 내용                 |
|--------|------------------------|
| 373    | MOVD에 사용할 수 없는 자료형입니다. |

## 적용 예제

MOVD #W1 DW1 W10 ; - #W1에서 워드 10개를 DW1부터 10개를 복사

**Set Bit( SETB )****[ 프로그램 입력 형식 ]**

|                         |
|-------------------------|
| SETB #W1.00 W4 L1 D15 ; |
|-------------------------|

**[ 프로그램 입력 설명 ]**

|        |                  |
|--------|------------------|
| SETB   | - Set Bit 지령 입력  |
| #W1.00 | - 값을 기록할 레지스터 입력 |
| W4     | - 폭 입력           |
| L1     | - 반복 회수 입력       |
| D15    | - 기록할 값 입력       |

레지스터의 특정 비트에 값을 기록합니다.

즉, 레지스터를 구성하는 전체 비트들 중에서 일부 비트들에 값을 기록합니다.

**[ 입력 인자 자료형 ]**

|          | 비트( B ) | 정수( W ) | 실수( F ) | 상수 |
|----------|---------|---------|---------|----|
| 기록될 레지스터 | O       | X       | X       | X  |
| 폭        | X       | O       | X       | X  |
| 반복 회수    | X       | O       | X       | X  |
| 값        | X       | O       | X       | X  |

| 이벤트 코드 | 이벤트 내용                 |
|--------|------------------------|
| 376    | SETB에 사용할 수 없는 자료형입니다. |

**적용 예제**

SETB #W1.00 W4 L1 D15 ;    - #W1.00에서 #W1.03까지 값 15을 1번 반복하여 저장

## Set Data( SETD )

## [ 프로그램 입력 형식 ]

|                   |
|-------------------|
| SETD #W1 W4 D15 ; |
|-------------------|

## [ 프로그램 입력 설명 ]

|      |                  |
|------|------------------|
| SETD | - Set Data 지령 입력 |
| #W1  | - 값을 기록할 레지스터 입력 |
| W4   | - 폭 입력           |
| D15  | - 기록할 값 입력       |

지정된 범위의 레지스터를 지정한 값으로 초기화 합니다.

## [ 입력 인자 자료형 ]

|          | 비트( B ) | 정수( W ) | 실수( F ) | 상수 |
|----------|---------|---------|---------|----|
| 기록될 레지스터 | X       | O       | X       | X  |
| 폭        | X       | O       | X       | X  |
| 값        | X       | O       | X       | X  |

| 이<br>벤트<br>코드 | 이벤트 내용                  |
|---------------|-------------------------|
| 375           | SHIFT에 사용할 수 없는 자료형입니다. |

|       |
|-------|
| 적용 예제 |
|-------|

SETD #W1 W4 D15 ;      - #W1에서 #W4까지 값 15을 저장

---

# 8

## 보조 기능

## 8.1 일반적인 M 코드

이동지령과 M 코드를 동일 블록 내에 지령하는 경우에

1. 이동지령과 보조지령을 동시에 개시할 것인지,
2. 이동지령이 종료된 후 M 기능 지령을 개시할 것인지 등은 Plc Program에서 결정합니다.

일반적으로 하나의 블록에는 M 코드를 최대 10까지 사용 가능합니다.

| M 코드 | 기능          | 의미                                                                              |
|------|-------------|---------------------------------------------------------------------------------|
| M02  | 프로그램 종료     | 프로그램의 종료를 나타내는 지령으로, 그 블록의 동작이 완료된 후 정지합니다.<br>M30 과 동일한 동작으로 모든 지령이 리셋 처리 됩니다. |
| M03  | 정 회전[ CW ]  | 이 지령 전에 기어변속, 회전수를 미리 조정해야 합니다.                                                 |
| M04  | 역 회전[ CCW ] | 이 지령 전에 기어변속, 회전수를 미리 조정해야 합니다.                                                 |
| M05  | 정지          | 회전방향을 바꿀 때나 기어 변속 시에 사용합니다.                                                     |
| M30  | 프로그램 종료     | 프로그램 종료됨과 동시에 프로그램은 다시 첫머리로 복귀 합니다. 모든 지령이 RESET 처리 됩니다.                        |
| M98  | 보조 프로그램 호출  | 자동프로그램을 진행 중에 보조 프로그램 호출하는 기능                                                   |
| M99  | 보조 프로그램 종료  | 보조프로그램을 종료하는 기능.<br>[ 메인 프로그램을 계속 반복 진행할 경우에도 사용 ]                              |



## 8.2 코드에 의한 서브 프로그램 호출

매크로 프로그램 호출을 위해 사용자가 원하는 M 코드를 지정할 수 있습니다.

매크로 프로그램을 호출하는 번호와 이름은 P6223-6242와 P6243-6282에서 설정합니다.

매크로 프로그램에 사용중인 Local 인수( #W1 ~ #W33 )의 지령도 가능하며, 내부에서 새로운 변수의 사용도 가능합니다.

```
SCALL(M98) P_
```

위의 지령을 아래와 같이 지령할 수 있습니다.

```
M_
SCALL(M98) - 서브 프로그램 호출 M 코드
P_ - 호출할 프로그램 이름(번호)
M_ - 파라미터에 설정된 서브 프로그램 호출 M 코드
```

일반적인 M 코드 지령 외에 멀티 M 코드 지령이 있습니다.

Mxx = xxxx 형태로 지령하며 M 코드 지령과 같은 역할을 수행합니다.

또한 Mx = xxxx 형태로 지령되면 F 레지스터에 다음과 같이 저장됩니다.

|         | M Code No | M Code Data |
|---------|-----------|-------------|
| M1 = 03 | 1         | 3           |

[ 관련 파라미터 ]

| 번호          | 내용                 |
|-------------|--------------------|
| F102 - F109 | - M Code No 를 저장   |
| F110 - F117 | - M Code Data 를 저장 |

### 8.3 동기 M 코드 설정

채널 1과 채널 2를 가공 도중에 대기하고 싶을 때는 M 코드로 제어합니다.

각각의 채널 Program에 대기용 M 코드를 지령함으로써, 지령한 블록에서 2개의 채널을 대기하게 할 수 있습니다. 자동 운전 중에 한쪽의 채널에서 대기용 M 코드가 지령 되면, 다른 쪽의 채널에서 동일한 M 코드가 지령 되는 것을 기다리고 있다가, 다른 블록의 실행을 개시합니다. 대기의 M 코드로서 사용하는 M 코드의 범위를 사전에 파라미터 P6055 – P6056에 설정하여 놓습니다.

| 채널 1의 프로그램         | 채널 2의 프로그램            |                        |
|--------------------|-----------------------|------------------------|
| MOV X0 Y0 F1000 ;  | MOV X0 Y0 F1000;      |                        |
| X50 Y-50 ;         | X100 Y100 ;           |                        |
| M21 ;              | M21 ;                 | 대기                     |
| LIN X100 Y0 F500 ; | LIN X50 Y50 F600 ;    | 채널 1과 채널 2의<br>동시 독립운전 |
| :                  | :                     |                        |
| :                  | M22 ;                 |                        |
| :                  | 대기( M22 )             | 대기                     |
| M22 ;              | LIN X100 Y100 F1000 ; | 채널 2만 운전               |
| M23 ;              | :                     |                        |
| 대기( M23 )          | :                     |                        |
| MOV X0 Y0 F1000 ;  | M23 ;                 | 대기                     |
| X100 Y100 ;        | MOV X150 Y0 F1000 ;   | 채널 1과 채널 2의<br>동시 독립운전 |
| :                  | X100 Y100 ;           |                        |
| M24 ;              | :                     |                        |
| 대기( M24 )          | :                     |                        |
| PEND ;             | M24 ;                 | 대기                     |
|                    | PEND ;                | 프로그램 종료                |

## [ 관련 파라미터 ]

| 번호    | 내용              |
|-------|-----------------|
| P6052 | 동기 M Code 사용 여부 |
| P6055 | 동기 M Code 시작 번호 |
| P6056 | 동기 M Code 종료 번호 |

| 이벤트 코드 | 이벤트 내용                         |
|--------|--------------------------------|
| 396    | 설정되지 않은 채널에서 동기 M 코드가 입력되었습니다. |
| 397    | 입력된 동기 M 코드가 서로 다릅니다.          |
| 398    | 입력된 동기 M 코드가 범위에 있지 않습니다.      |

|       |
|-------|
| 적용 예제 |
|-------|

ABS MOV F = 1000 ;

N1 FOR[ #W1=0,#W1 LE 10,5 ] ;

X[ #W1 ] ;

FEND ;

X0 ;

M21 ;

- 동기 M Code 설정

GOTO N1 ;

M30 ;

ABS MOV F = 1000 ;

N1 FOR[ #W1=0,#W1 LE 10,5 ] ;

X[ #W1 ] ;

FEND ;

M21 ;

- 동기 M Code 설정

X0 ;

GOTO N1 ;

M30 ;



---

# 9

## 매크로 프로그램

## 9.1 매크로 호출 명령어

### 단순 호출

[ 프로그램 입력 형식 ]

```
MCALL(G65) P_ L_ < 인수 지정 > ;
```

[ 프로그램 입력 설명 ]

|              |           |
|--------------|-----------|
| MCALL( G65 ) | - 매크로 호출  |
| P_           | - 프로그램 번호 |
| L_           | - 반복 횟수   |

커스텀 매크로의 최대 특징은

1. 변수가 사용될 수 있습니다.
2. 변수들간의 연산이 가능합니다.
3. 매크로 수행에 의해 변수에 실제 값을 설정 할 수 있습니다.
4. 변수는 변수 번호에 따라 Local 변수, Common 변수와 System 변수로 분류됩니다.
5. 변수는 정수, 실수, 비트 형태의 값을 갖습니다.
6. 프로그램 번호는 4자리 수만 가능합니다.
7. #[0]은 인자 사용여부 플래그 정보  
Ex ] #B0.2가 0이면 3번째 인자가 호출 시에 사용되지 않았음을 의미합니다.
8. UL 변수를 사용하고, 호출한 프로그램의 하위 서브 프로그램의 영역을 사용합니다.  
( Macro 프로그램과 서브 프로그램은 결국 동일, 마지막 서브 프로그램에서는 Macro 프로그램을 호출할 수 없습니다. )
9. 로컬 변수 사용( # )  
파라미터에 설정된 CH의 개수에 의해 채널에서 사용할 수 있는 로컬변수의 개수가 정해집니다.  
Ex ] 파라미터에 채널 개수 2라면 채널1은 1000개의 변수를 채널2은 1000개의 변수를 사용할 수 있습니다. 채널 1에서 1000개의 변수 중에서 또 서브 프로그램까지 지원하기 위해서 1000개 중에서 나눠야 하며 서브 프로그램 개수만큼 나눠야 합니다.

## ① 인수 지정 방법 1.

1. 어드레스 **G, L, N, P, O**를 제외한 모든 어드레스에 할당 가능합니다.

(A\_ B\_ C\_ ... Z\_ )

2. 일반적으로 알파벳 지령 순서는 상관없지만 **I, J, K** 만은 알파벳 순서에 맞게 지령해야 합니다.

B\_ A\_ D\_ L\_ K\_ ;                      - 정상

B\_ A\_ D\_ J\_ L\_ ;                      - 비정상

[ 인자 할당 방법 1. 에 따른 어드레스( **G, L, N, P, O** 제외 ) ]

| 어드레스 | 커스텀 매크로 본체의 변수 |
|------|----------------|
| A    | #F1            |
| B    | #F2            |
| C    | #F3            |
| D    | #F7            |
| E    | #F8            |
| F    | #F9            |
| H    | #F11           |
| I    | #F4            |
| J    | #F5            |
| K    | #F6            |
| M    | #F13           |
| Q    | #F17           |
| R    | #F18           |
| S    | #F19           |
| T    | #F20           |
| U    | #F21           |
| V    | #F22           |
| W    | #F23           |
| X    | #F24           |
| Y    | #F25           |
| Z    | #F26           |

## ② 인수 지정 방법 2.

A\_B\_C\_I\_J\_K\_I\_J\_K\_...

1. 어드레스 A, B, C 로 인수를 지정할 수 있고, 어드레스 I, J, K 로 1조가 되는 인수를 최대 10조까지 지정할 수 있습니다.
2. 같은 어드레스로 여러 개의 변수가 지정될 때는 결정된 순으로 지정해야 합니다.
3. 지정할 필요가 없는 어드레스는 생략할 수 있습니다.

[ 인수 지정 2. 로 지정하는 어드레스와 매크로 내에서 사용하는 변수의 변수 번호는 다음과 같이 대응됩니다. ]

| 어드레스 | 커스텀 매크로 본체의 변수 |
|------|----------------|
| A    | #F1            |
| B    | #F2            |
| C    | #F3            |
| I1   | #F4            |
| J1   | #F5            |
| K1   | #F6            |
| I2   | #F7            |
| J2   | #F8            |
| K2   | #F9            |
| I3   | #F10           |
| J3   | #F11           |
| K3   | #F12           |
| I4   | #F13           |
| J4   | #F14           |
| K4   | #F15           |
| I5   | #F16           |
| J5   | #F17           |
| K5   | #F18           |
| I6   | #F19           |
| J6   | #F20           |
| K6   | #F21           |
| I7   | #F22           |
| J7   | #F23           |
| K7   | #F24           |
| I8   | #F25           |



|    |      |
|----|------|
| J8 | #F26 |
| K8 | #F27 |
| I9 | #F28 |
| J9 | #F29 |
| K9 | #F30 |

I, J, K 뒤의 숫자 1 ~ 10 은 지령 된 순서를 나타내는 것으로 실제로는 지령하지 않습니다.

③ 인수 지정 1, 2 의 혼재

G65 의 블록 인수에 인수 지정 1, 2 타입의 인수가 있어도 알람이 발생하지 않습니다. 같은 인수에 대응하는 1 의 인수와 2 의 인수가 두 가지 방법에 의해 모두 지정되어 있으면 Type 1에 따른 지령이 유효합니다.

G65 A1.0 B2.0 I3.0 I4.0 D5.0 P1000 ;

#F1 = 1.0( A )

#F2 = 2.0( B )

#F3 =

#F4 = 3.0( I1 )

변수:

#F5 =

#F6 =

#F7 = 5.0( A ) [ #F7에는 4.0( I2 )보다 Type 1에 따르는 D5.0( D=#F7 )이 유효 ]

## ML 코드에 의한 매크로 호출

N\_ MCALL P\_ < 인수지정 > ;    - 방법을 대신하여 간단하게

N\_ ABCDEFGH < 인수지정 > ;    - 으로 지령할 수 있습니다.

ML를 제외한 최대 8자의 ML를 작성하여 최대 20 개의 ML를 매크로 프로그램 호출에 사용할 수 있습니다.

Ex] ABCDEFGH를 이용하여 Macro 프로그램을 호출하는 방법

|               |                                        |
|---------------|----------------------------------------|
| Macro 프로그램 번호 | 8888                                   |
| Macro 프로그램 이름 | ABCDEFGH                               |
| 모션 프로그램       | ABCDEFGH A1.0 B2.0 C3.0 I2.0 J3.0 K4.0 |

Ex] G100100를 이용하여 Macro 프로그램을 호출하는 방법

|               |                                       |
|---------------|---------------------------------------|
| Macro 프로그램 번호 | 9999                                  |
| Macro 프로그램 이름 | G100100                               |
| 모션 프로그램       | G100100 A1.0 B2.0 C3.0 I2.0 J3.0 K4.0 |

[ 관련 파라미터 ]

| 번호            | 내용            |
|---------------|---------------|
| P6223 - P6242 | Macro 프로그램 번호 |
| P6243 - P6282 | Macro 이름      |

- 주의 -

1. MCALL는 사용 불가능 합니다.
2. 호출된 매크로 프로그램 내에서는 사용할 수 없습니다.

## SCALL과 MCALL의 차이점

1. MCALL(G65)는 인수 지정이 가능합니다.
2. SCALL(M98)은 같은 블록내의 다른 M 코드, P 또는 L을 수행하고 서브 프로그램으로 Branch 됩니다. MCALL(G65)는 Branch 기능으로만 쓰입니다.
3. SCALL(M98)은 블록내의 O, N, P, L이 있는 경우 Single Stop에 의해 정지됩니다. MCALL(G65)는 정지하지 않습니다.
4. G65는 Local 변수의 Level를 변화시킬 수 있지만, SCALL(M98)은 그렇지 않습니다. 즉, MCALL 지령 이전에 #Wi가 정의되어 있었다라고 MCALL에 의해 호출된 프로그램내의#Wi는 이전의 #Wi와 전혀 다른 변수입니다.  
SCALL 이전에 #Wi가 정의되어 있는 경우에는 SCALL에 의해 서브 프로그램을 호출하더라도 #Wi는 같은 변수입니다.

## 다중 호출

|                             |
|-----------------------------|
| MMON( G66 ) P_ L_ < 인수 지정 > |
|-----------------------------|

### 1. 다중 호출도

1. 서브 프로그램 마찬가지로 매크로 프로그램 중에 매크로 프로그램을 호출할 수 있습니다.( 모달 호출은 메인에서만 호출 가능 ) 최대 4단계까지 호출 가능

### 2. 다중 모달 호출

1. 다중 모달 호출에서는 이동지령( Motion )을 실행할 때 마다 지정된 매크로가 호출됩니다. 모달인 매크로가 다중으로 지정되어 있는 경우에는 처음 매크로의 이동지령( Motion )을 실행될 때마다 다음 매크로를 호출합니다. 매크로는 지정된 것부터 차례로 호출됩니다.
2. 매크로 모달에 의해 호출된 프로그램에서 또다시 매크로 모달 호출( G66 )은 지원하지 않습니다.

## 적용 예제

“ 메인 프로그램

MMON( G66 ) P9100 ;

Z1000 ;                    - ( 1 - 1 )

MMON( G66 ) P9200 ;

Z15000 ;                    - ( 1 - 2 )

MMOF( G67 ) ;            - P9200 Cancelled

MMOF( G67 ) ;            - P9100 Cancelled

Z-25000 ;                    - ( 1 - 3 )

“ 9100

X5000 ;                    - ( 2 - 1 )

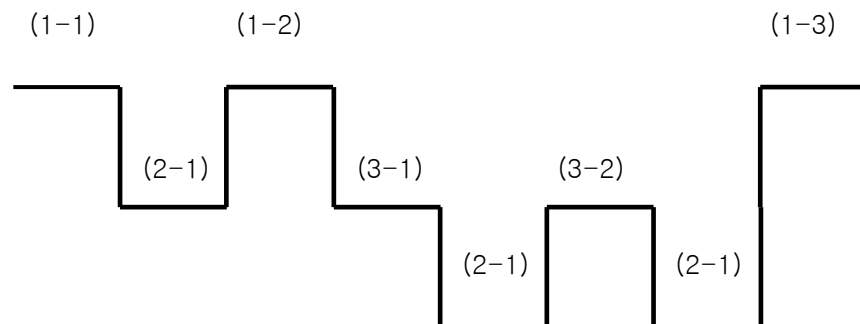
LOOP( M99 ) ;

“ 9200

Z6000 ;                    - ( 3 - 1 )

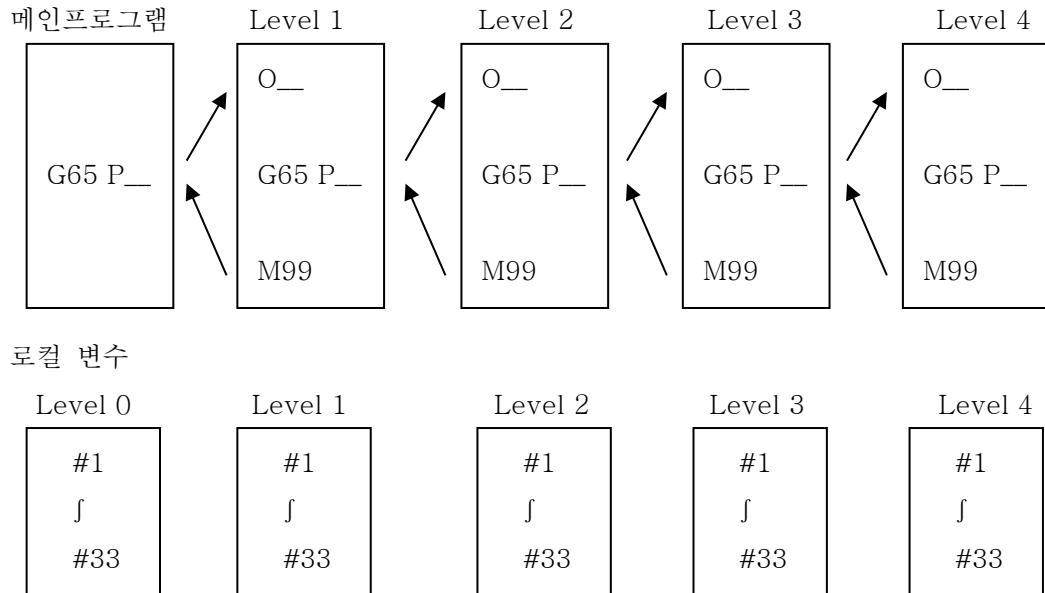
Z7000 ;                    - ( 3 - 2 )

LOOP( M99 ) ;



### 3. 커스텀 매크로 Level과 Local 변수

G65, G66 또는 매크로 프로그램 호출하는 G 코드에 의해 매크로가 호출될 때, 매크로의 Level이 한단계씩 증가됩니다.( Level 0 ~ Level 4 ). Local Variable의 Level도 1단계씩 증가합니다.



메인 프로그램( Level 0 )에 #1 ~ #33 의 로컬 변수가 있습니다.

G65 등에 따라 매크로( Level 1 )가 호출되면 메인 프로그램의 로컬 변수는 저장( Store )되고 Level 1의 매크로 프로그램을 위한 새로운 로컬 변수 #1 ~ #33 ( Level 1 )이 준비 됩니다.

매크로가 호출( Level 2, 3, 4 )될 때마다 로컬 변수( Level 1, 2, 3 )가 저장되고 다음 Level의 새로운 로컬 변수가 준비됩니다.

M99로 각 매크로에서 복귀하면 Level 2, 3에서 저장된 로컬 변수( Level 0, 1, 2, 3 )가 값이 보존된 채로 준비 됩니다.

## 9.2 커스텀 매크로 본체의 작성

### 매크로 프로그램 본체의 형식

1. 서브 프로그램과 같은 형식입니다.
2. Custom Macro 프로그램 내에서는 변수 사용 및 연산 지령, 제어 지령이 가능합니다.
3. 변수에 대응하는 실제 수치의 지정은 매크로 호출지령으로 합니다.( G65 A\_ B\_ ... )

### 변수의 사용방법

1. 변수란 매크로 중에서 직접 수치를 주는 대신 변수로 지정대두고 매크로를 호출하는 그때 그때에 따라 변수의 값을 주어서 매크로 프로그램에 융통성, 범용성을 주는 것입니다.
2. 변수는 여러 개 사용될 수 있고, 변수 번호로 식별합니다.

#### ① 변수의 표현

#Wi - 정수형 변수 [ i = 1, 2, 3, ... ]  
 #Fi - 실수형 변수 [ i = 1, 2, 3, ... ]  
 #Bi.j - 비트형 변수 [ i = 1, 2, 3, ... j = 1, 2, 3, ... ]

특정 레지스터에 변수를 지령하고자 할 때는 다음과 같이 입력합니다.

레지스터 자료형 변수 번호

Ex] FW1 = 1;

F - 레지스터를 의미

- B: Bit를 의미

W - W: Word를 의미

- F: Float를 의미

1 - 값을 저장하거나 읽을 변수 번호를 의미

#### ② 변수의 활용

1. 어드레스에 이어지는 수치를 변수로 대신할 수 있습니다.  
 어드레스 #W1, 어드레스 -#W1로 프로그램이 가능합니다.
2. 변수번호를 변수로 치환하는 방법( 간접변수 )  
 만약, #W100 = 105이고 #W105 = -500인 경우, “##W100” 이란 표현은 사용하지

않고 대신 “#W[#W100]”이라고 합니다.

3. 어드레스 :, N은 활용이 되지 않습니다.
4. 어드레스마다 지정된 최대 지령치를 넘는 값은 지령할 수 없습니다.
5. [ 식 ]에서는 모든 변수가 실수형으로 연산 됩니다.

### ③ 미정의 변수

1. 아직 정의 되지 않은 변수의 값은 0 입니다.

입력: ABS(G90) X100 Z#W1 ;

해석: ABS(G90) X100 Z0. ;

#W1 = 0 일 때      #W2 = #W1 ;

입력:                    #W2 = #W1 \* 5 ;

해석:                    #W2 = 0 ;

                            #W2 = 0 \* 5 ;



### 9.3 프로그램 예제

적용 예제 - 매크로 프로그램 Plane Drill

```

REF(G28) INC(G91) Z0 ;
REF(G28) X0 Y0 ;
ABS(G90) WCS(G92) X150 Y150 Z200 ;
Z50 ;
MOV(G00) X0 Y0 ;
#F100 = 15 ; - X Drill Number
#F102 = 10 ; - X Distance
#F103 = 0 ; - X Count
#F111 = 5 ; - Y Line Count
#F112 = 10 ; - Y Distance
#F113 = 0 ; - Y Count
#F200 = 0 ; - Start X Pos
#F201 = 0 ; - Start Y Pos
ABS(G90) X[#F200] Y[#F201] ;
N10 ;
ABS(G90) X[#F200] Y[#F201+#F112*#F113] ;
N20 ;
ABS(G90) X[#F200+#F102*#F103] Y[#F201+#F112*#F113] ;
LIN(G1) Z-10 F100 ;
MOV(G0) Z5 ;
#F103 = #F103+1 ;
IF [#F103 LT #F100-1] GOTO N20 ;
#F103 = 0 ;
#F113 = #F113 + 1 ;
IF [#F113 LT #F111-1] GOTO N10 ;
INC(G91) MOV(G0) Y50 ;
X50 ;
#F102 = #F102 + 1 ;
PEND ;

```

## 적용 예제 - 원호가공 ( 1도씩 분할 )

```
REF(G28) INC(G91) Z0 ;
REF(G28) X0 Y0 ;
ABS(G90) WCS(G92) X150 Y150 Z200 ;
Z50 ;
MOV(G0) X0 Y0 ;
#F100 = 0 ; - 시작 Angle
#F101 = 50 ; - 원호 Radius
N20
LIN(G1) X[SIN[#F100]*#F101] Y[COS[#F100]*#F101] ;
#F100 = #F100 + 1 ;
IF [#F100 LE 360.0] GOTO N20 ;
INC(G91) MOV(G0) Y50 ;
X50 ;
#F102 = 1 ; - Count
#F102 = #F102 + 1 ;
PEND ;
```

## 적용 예제 – While – Wend 예제

```

REF(G28) INC(G91) Z0 ;
REF(G28) X0 Y0 ;
ABS(G90) WCS(G92) X150 Y150 Z200 ;
MOV(G0) Z10 ;
MOV(G0) X0 Y0 ;
#F100 = 0 ; – Start Angle
#F101 = 50 ; – Radius
#F102 = 45 ; – Between Angle
N150 WHILE[#F100 LE [360.-#F102]] DO 210 ;
N200 WHILE[#F101 GE 10.] DO 220 ;
MOV(G0) X[SIN[#F100]*#F101]Y[COS[#F100]*#F101] ;
LIN(G1) Z-20 F100 ;
MOV(G0) Z10 ;
#F101 = #F101 - 10 ; – Increase
WEND 220 ;
#F100 = #F100+#F102 ; – Radius Decrease
#F101 = 50 – Radius
WEND 210 ;
INC(G91) MOV(G0) Y50 ;
X50 ;
PEND ;

```



---

# 10

## 매크로 프로그램

## 10.1 매크로 호출 명령

### 10.1.1 빠른 명령

[ 입력 유형]

|                                |
|--------------------------------|
| MCALL( G65 ) P_ L_ < 인자 지정 > ; |
|--------------------------------|

[ Program Input Description ]

|              |           |
|--------------|-----------|
| MCALL( G65 ) | - 매크로 호출  |
| P_           | - 프로그램 번호 |
| L_           | - 반복 횟수   |

커스텀 매크로의 특징은 다음과 같습니다.

1. 변수를 사용할 수 있습니다.
2. 변수 간의 산술 연산이 가능합니다.
3. 매크로를 통해 변수에 실제 값을 설정할 수 있습니다.
4. 변수는 변수 번호에 따라 지역변수, 공통변수, 시스템변수로 구분됩니다.
5. 변수 유형에는 정수, 부동 소수점 숫자 및 비트가 있습니다.
6. 프로그램 번호는 네 자리여야 합니다.
7. #[0]은 인자(Argument)사용 여부에 대한 플래그 정보입니다.  
예시] 만약 #B0.2가 0이면, 호출 시 세번째 인자가 사용되지 않았음을 의미합니다.
8. UL변수 및 호출된 프로그램의 부프로그램(Sub-program)영역을 사용합니다.  
(매크로 프로그램과 부프로그램은 동일합니다. 마지막 부프로그램은 매크로 프로그램을 호출할 수 없습니다.)
9. 지역 변수가 사용됩니다.( # )  
채널 내에서 사용 가능한 지역 변수의 수는 파라미터에서 설정된 채널 수에 따라 달라집니다.  
예시] 파라미터에서 채널 수가 2개로 설정된 경우, 채널 1은 1,000개의 변수를 사용할 수 있고, 채널 2도 1,000개의 변수를 사용할 수 있습니다. 채널 1 내의 1,000개의 변수와 부프로그램과의 호환성을 유지하기 위해, 1,000을 부프로그램의 수로 나누어 할당해야 합니다.

## ① 인자 지정 방법1.

1. G, L, N, P, O를 제외한 모든 어드레스(Address)에 할당할 수 있습니다.

(A\_ B\_ C\_ ... Z\_ )

2. 일반적으로 알파벳 지시 순서는 상관없으나, I,J,K는 반드시 알파벳 순서대로 지시해야 합니다..

B\_ A\_ D\_ L\_ K\_ ;                    - 정상

B\_ A\_ D\_ J\_ L\_ ;                    - 비정상

[ 인자 할당 방법1에 따른 어드레스 .( G, L, N, P, O 제외) ]

| Address | Variables of custom macro body |
|---------|--------------------------------|
| A       | #F1                            |
| B       | #F2                            |
| C       | #F3                            |
| D       | #F7                            |
| E       | #F8                            |
| F       | #F9                            |
| H       | #F11                           |
| I       | #F4                            |
| J       | #F5                            |
| K       | #F6                            |
| M       | #F13                           |
| Q       | #F17                           |
| R       | #F18                           |
| S       | #F19                           |
| T       | #F20                           |
| U       | #F21                           |
| V       | #F22                           |
| W       | #F23                           |
| X       | #F24                           |
| Y       | #F25                           |
| Z       | #F26                           |

## ② 인자 할당 방법2.

A\_B\_C\_I\_J\_K\_I\_J\_K\_...

1. 인자는 기본 어드레스(A, B, C)와 \*\*반복 어드레스(I, J, K)\*\*의 조합으로 구성됩니다. 특히 I, J, K는 I1, J1, K1에서 I10, J10, K10까지 최대 10개 세트로 확장하여 지정할 수 있습니다.
2. 동일한 어드레스로 여러 변수를 지정하는 경우, 결정된 시점의 순서대로 지정해야 합니다.
3. 지정할 필요가 없는 어드레스는 생략할 수 있습니다.

[인자 지정2 : 매크로에서 사용되는 어드레스와 변수 번호의 매칭.]

| 어드레스 | 커스텀 매크로 본체의 변수 |
|------|----------------|
| A    | #F1            |
| B    | #F2            |
| C    | #F3            |
| I1   | #F4            |
| J1   | #F5            |
| K1   | #F6            |
| I2   | #F7            |
| J2   | #F8            |
| K2   | #F9            |
| I3   | #F10           |
| J3   | #F11           |
| K3   | #F12           |
| I4   | #F13           |
| J4   | #F14           |
| K4   | #F15           |
| I5   | #F16           |
| J5   | #F17           |
| K5   | #F18           |
| I6   | #F19           |
| J6   | #F20           |
| K6   | #F21           |
| I7   | #F22           |
| J7   | #F23           |
| K7   | #F24           |
| I8   | #F25           |



|    |      |
|----|------|
| J8 | #F26 |
| K8 | #F27 |
| I9 | #F28 |
| J9 | #F29 |
| K9 | #F30 |

I, J, K 뒤에 붙는 1~10까지의 숫자는 인자의 지정 순서를 나타내는 것이며, 실제로 입력(명령)하는 숫자가 아닙니다.

### ③ 인자 지정 방법 1과 2의 혼용

하나의 G65 블록 내에 방법 1과 방법 2의 인자 형식이 섞여 있어도 알람(에러)이 발생하지 않습니다. 만약 방법 1과 방법 2가 동일한 변수를 중복으로 지정하게 될 경우에는, 방법 1 (Type 1)에 의한 지시가 우선적으로 유효합니다..

G65 A1.0 B2.0 I3.0 I4.0 D5.0 P1000 ;

#F1 = 1.0( A )

#F2 = 2.0( B )

#F3 =

변수: #F4 = 3.0( I1 )

#F5 =

#F6 =

#F7 = 5.0( A ) [ #F7, D5.0( D=#F7 ) 타입1 지정방식이 4.0( I2 ) 보다 우선 ]

### 10.1.2 ML 코드에 의한 매크로 호출

1. 매크로 프로그램 9010.mp ~ 9029.mp를 호출하는 G 코드는 파라미터 P4650~4689에서 설정할 수 있습니다..

N\_ MCALL P\_ < 인자 지정 > ;                      - 이 방식 대신 아래처럼 지시할 수 있습니다.  
 N\_ ABCDEFGH < 인자 지정 > ;                      - G코드xx와 해당 매크로 프로그램은 파라미터 P4650~4689에서 설정 가능합니다.

2. ML을 최대 8자(ML 제외)까지 준비하면, 최대 20개의 ML을 사용하여 매크로 프로그램(9010.mp ~ 9029.mp)을 호출할 수 있습니다.

- 주의 사항 -

1. MCALL은 사용할 수 없습니다.
2. 호출된 매크로 프로그램 내에서는 이 기능을 사용할 수 없습니다.

### 10.1.3 G코드에 의한 매크로 호출

1. 매크로 프로그램 9010.mp ~ 9029.mp를 호출하는 G 코드는 파라미터 P4690~4909에서 설정할 수 있습니다.

N\_ MCALL P\_ < 인자 지정 > ;                      - 이 방식 대신 아래와 같이 지시 가능합니다.  
 N\_ Gxx < 인자 지정 > ;                      - G코드xx와 해당 매크로 프로그램은 파라미터 P4690~P4909에서 설정 가능합니다.

2. G00을 제외하고, G01 ~ G255 중 최대 20개의 G 코드를 매크로 프로그램(9010.mp ~ 9029.mp) 호출용으로 사용할 수 있습니다. 또한 G 코드는 소수점 첫째 자리까지 사용할 수 있습니다

- 주의사항 -

1. G65는 사용할 수 없습니다.
2. 호출된 매크로 프로그램 내에서는 이 기능을 사용할 수 없습니다.

#### 10.1.4 M코드에 의한 매크로 호출

1. 매크로 프로그램 9010.mp ~ 9029.mp를 호출하는 M 코드는 파라미터 P4710~4729에서 설정할 수 있습니다.

N\_ MCALL P\_ < 인자 지정 > ;

- 아래와 같이 지시 가능합니다.

N\_ Mxx < 인자 지정 > ;

- M코드 xx와 해당 매크로 프로그램은  
파라미터 P4710~4729에서 설정 가능합니다.

2. M FIN(M 코드 완료 신호) 지시 신호는 출력되지 않습니다. (SCALL 및 LOOP와 동일)
3. M01 ~ M97 중 최대 10개의 M 코드를 사용할 수 있습니다.  
(단, PEND는 사용할 수 없습니다.)

만약 G 코드나 M 코드에 의해 보조 프로그램(Sub-program)을 호출하는 도중에 이 MXX가 지시되면, 이는 일반 M 코드로 처리됩니다.

이러한 M 코드들은 일반적인 M 코드와 다르며, 블록의 시작 부분(또는 NL 뒤)에서 지시됩니다..

### 10.1.5 SCALL 과 MCALL의 차이점

1. MCALL(G65)은 인자(Factor)를 지정할 수 있습니다.
2. SCALL(M98)은 동일 블록 내의 다른 M 코드, P 또는 L을 실행하며  
부프로그램(Sub-program)으로 분기합니다. 반면, MCALL(G65)은 오직 분기  
기능으로만 사용됩니다.
3. 블록 내에 O, N, P, L이 있는 경우, SCALL(M98)은 싱글 스톱(Single Stop)에 의해  
정지되지만, MCALL(G65)은 정지되지 않습니다.
4. G65는 지역 변수(Local variables)의 레벨을 변경할 수 있지만, \*\*SCALL(M98)\*\*은  
변경할 수 없습니다.  
즉, MCALL 지시 이전에 #Wi가 정의되어 있더라도, MCALL에 의해 호출된 프로그램  
내의 #Wi는 이전의 #Wi와는 완전히 다른 변수입니다.  
만약 SCALL 이전에 #Wi가 정의되었다면, SCALL에 의해 부프로그램이  
호출되더라도 #Wi는 동일한 변수로 취급됩니다..

### 10.1.6 다중 호출

|                             |
|-----------------------------|
| MMON( G66 ) P_ L_ < 인자 지정 > |
|-----------------------------|

#### 1. 다중 호출 레벨

1. 부프로그램과 마찬가지로, 매크로 프로그램 내에서 다른 매크로 프로그램을 호출할  
수 있습니다. (단, 모달 호출(Modal call)은 메인 프로그램에서만 사용 가능합니다.)
2. 최대 4단계 까지 호출이 가능합니다.

#### 2. 다중 모달 호출

1. 다중 모달 호출 상태에서는 이동 명령(Motion instruction)이 실행될 때마다 지정된  
매크로가 호출됩니다. 만약 모달 매크로가 여러 레이어(층)에 지정되어 있다면, 첫  
번째 매크로의 이동 명령이 실행될 때 다음 단계의 매크로가 호출됩니다.  
매크로는 지정된 순서대로 순차적으로 호출됩니다.
2. 이미 매크로 모달에 의해 호출된 프로그램 내에서는 매크로 모달 호출(G66)을 다시  
사용할 수 없습니다.

## 적용 예시

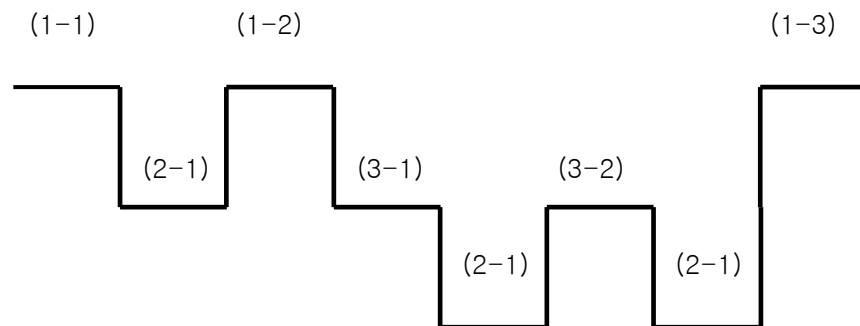
```

“ main program
MMON(G66) P9100 ;
Z1000 ; - (1 - 1)
MMON(G66) P9200 ;
Z15000 ; - (1 - 2)
MMOF(G67) ; - P9200 매크로 호출 취소
MMOF(G67) ; - P9100 매크로 호출 취소
Z-25000 ; - (1 - 3)

“ 9100
X5000 ; - (2 - 1)
LOOP(M99) ;

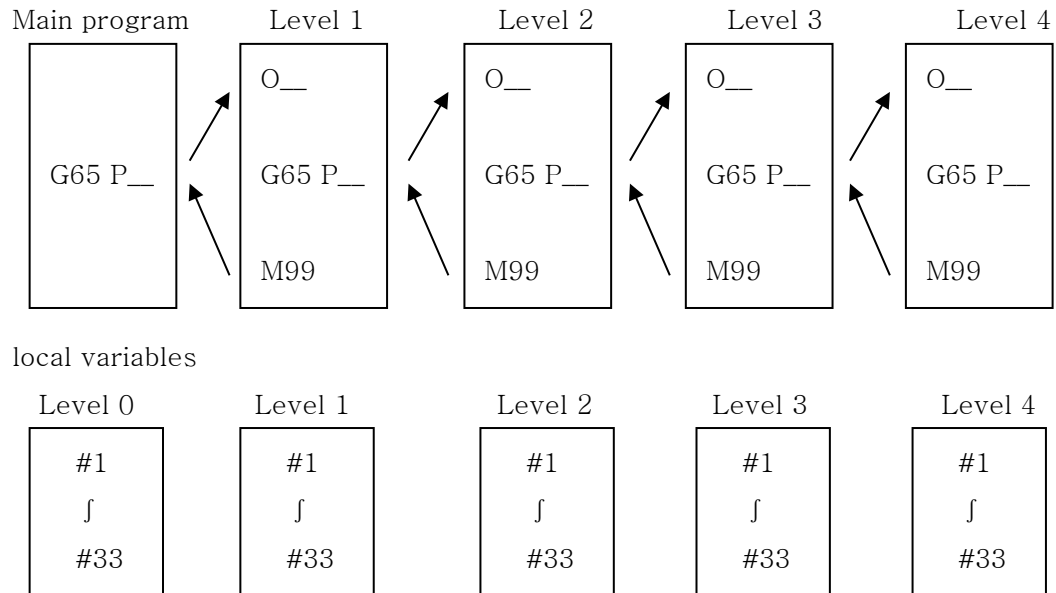
“ 9200
Z6000 ; - (3 - 1)
Z7000 ; - (3 - 2)
LOOP(M99) ;

```



### 3. 커스텀 매크로 레벨과 지역 변수

G65, G66 또는 매크로 호출용 G 코드에 의해 매크로가 호출되면, 매크로 레벨은 한 단계씩 증가합니다. (레벨 0 ~ 레벨 4) 이와 동시에 지역 변수(Local Variable)의 레벨 또한 한 단계씩 함께 증가합니다.



#### [ 지역 변수의 레벨별 동작 규칙 ]

메인 프로그램(레벨 0)에는 #1 ~ #33의 지역 변수가 존재합니다.

G65 등으로 매크로(레벨 1)를 호출하면, 메인 프로그램에 있던 지역 변수들은 자동으로 저장(Store)되며, 레벨 1 매크로 프로그램을 위한 새로운 지역 변수 #1 ~ #33이 준비됩니다.

이후 매크로가 추가로 호출될 때마다(레벨 2, 3, 4), 이전 레벨의 지역 변수들은 저장되고 다음 레벨을 위한 새로운 지역 변수가 매번 준비됩니다.

M99를 통해 각 매크로에서 복귀하면, 이전에 저장되었던 해당 레벨의 지역 변수들이 기존에 가지고 있던 값들을 그대로 유지한 채 다시 불러와집니다.

## 10.2 커스텀 매크로 본체 준비

### 10.2.1 매크로 프로그램 본체의 형식

1. 매크로 프로그램의 형식은 부프로그램(Sub-program)과 동일합니다.
2. 커스텀 매크로 프로그램 내에서는 변수 사용, 산술 연산 명령 및 제어 명령을 사용할 수 있습니다.
3. 변수에 매칭되는 실제 수치는 매크로 호출 명령(예: G65 A\_ B\_ ...)에 의해 지정됩니다.

### 10.2.2 변수 사용 방법

1. 매크로 내에서 수치(데이터)를 직접 정의하는 대신 변수를 정의하여 사용합니다. 매크로가 호출될 때마다 해당 변수에 새로운 값이 부여되므로, 매크로 프로그램에 유연성과 범용성을 더해줍니다.
2. 여러 개의 변수를 동시에 사용할 수 있으며, 각 변수는 변수 번호를 통해 식별됩니다.

#### ① 변수의 표현식

|       |              |                                       |
|-------|--------------|---------------------------------------|
| #Wi   | - 정수형 변수     | [ i = 1, 2, 3, ... ]                  |
| #Fi   | - 부동 소수점형 변수 | [ i = 1, 2, 3, ... ]                  |
| #Bi.j | - 비트형 변수     | [ i = 1, 2, 3, ... j = 1, 2, 3, ... ] |

변수를 특정 레지스터에 지정할 때는 다음과 같이 입력합니다.

레지스터 데이터 타입 변수 번호 예시

FW1 = 1;

F - 레지스터를 의미

- B: 비트(Bit)를 의미

W - W: 워드(Word)를 의미

- F: 부동소수점을 의미

1 - 저장하거나 읽어올 변수 번호를 의미.

#### ② 변수의 사용

1. 어드레스 뒤에 오는 숫자는 변수로 대체할 수 있습니다.  
ex] Address #W1, Address -#W1

2. 변수 번호를 다른 변수로 대체하는 방법 (간접 지정).  
 $\#W100 = 105$  이고  $\#W105 = -500$  일때,  $\#\#W100$  과 같은 표기법은 사용할 수 없습니다. 대신  $\#[\#W100]$  형식을 사용해야 합니다.
3. 어드레스 /, :, O, N 은 변수로 대체하여 사용할 수 없습니다.  
 Optional Block Skip /n n(n = 1, 2, ... 9) 또한 변수로 사용할 수 없습니다.
4. 각 어드레스에 지정된 최대 지시 값 (허용 범위)를 초과하는 수치는 입력 할 수 없습니다.
5. 수식 내의 모든 변수는 float 형식으로 계산됩니다.

③ 정의되지 않은 변수

1. 정의되지 않은 변수는 0으로 취급됩니다.

|            |     |                      |
|------------|-----|----------------------|
|            | 입력: | ABS(G90) X100 Z#W1 ; |
|            | 해석: | ABS(G90) X100 Z0. ;  |
|            |     | #W2 = #W1 ;          |
| #W1 = 0일때, | 입력: | #W2 = #W1 * 5 ;      |
|            |     | #W2 = 0 ;            |
|            | 해석: | #W2 = 0 * 5 ;        |



## 10.3 프로그램 예제

|  |                        |
|--|------------------------|
|  | 적용 예시 : 평면 드릴 매크로 프로그램 |
|--|------------------------|

REF( G28 ) INC( G91 ) Z0 ;

REF( G28 ) X0 Y0 ;

ABS( G90 ) WCS( G92 ) X150 Y150 Z200 ;

Z50 ;

MOV( G00 ) X0 Y0 ;

#F100 = 15 ;

- X축 드릴 가공횟수

#F102 = 10 ;

- X 축 거리

#F103 = 0 ;

- X 축 카운트

#F111 = 5 ;

- Y축 라인 가공횟수

#F112 = 10 ;

- Y 축 거리

#F113 = 0 ;

- Y 카운트

#F200 = 0 ;

- 시작 X좌표

#F201 = 0 ;

- 시작 Y좌표

ABS( G90 ) X[#F200] Y[#F201] ;

N10 ;

ABS( G90 ) X[#F200] Y[#F201+#F112\*#F113] ;

N20 ;

ABS( G90 ) X[#F200+#F102\*#F103] Y[#F201+#F112\*#F113] ;

LIN( G1 ) Z-10 F100 ;

MOV( G0 ) Z5 ;

#F103 = #F103+1 ;

IF [ #F103 LT #F100-1 ] GOTO N20 ;

#F103 = 0 ;

#F113 = #F113 + 1 ;

IF [ #F113 LT #F111-1 ] GOTO N10 ;

INC( G91 ) MOV( G0 ) Y50 ;

X50 ;

#F102 = #F102 + 1 ;

PEND ;

|  |                        |
|--|------------------------|
|  | 적용 예시 - 원호 가공 매크로 프로그램 |
|--|------------------------|

REF( G28 ) INC( G91 ) Z0 ;

REF( G28 ) X0 Y0 ;

ABS( G90 ) WCS( G92 ) X150 Y150 Z200 ;

Z50 ;

MOV( G0 ) X0 Y0 ;

#F100 = 0 ;

- 시작 각도

#F101 = 50 ;

- 원호 반지름

N20

LIN( G1 ) X[SIN[#F100]\*#F101] Y[COS[#F100]\*#F101] ;

#F100 = #F100 + 1 ;

IF [ #F100 LE 360.0 ] GOTO N20 ;

INC( G91 ) MOV( G0 ) Y50 ;

X50 ;

#F102 = 1 ;

- 카운트

#F102 = #F102 + 1 ;

PEND ;

## 적용 예시 - WHILE-WEND문 예제

```

REF(G28) INC(G91) Z0 ;
REF(G28) X0 Y0 ;
ABS(G90) WCS(G92) X150 Y150 Z200 ;
MOV(G0) Z10 ;
MOV(G0) X0 Y0 ;
#F100 = 0 ;
#F101 = 50 ;
#F102 = 45 ;
N150 WHILE[#F100 LE [360.-#F102]] DO 210 ;
N200 WHILE[#F101 GE 10.] DO 220 ;
MOV(G0) X[SIN[F#100]*#F101]Y[COS[#F100]*#F101] ;
LIN(G1) Z-20 F100 ;
MOV(G0) Z10 ;
#F101 = #F101 - 10 ;
WEND 220 ;
#F100 = #F100+#F102 ;
#F101 = 50
WEND 210 ;
INC(G91) MOV(G0) Y50 ;
X50 ;
PEND ;

```

- 시작 각도
- 반지름
- 각도 간격
- 반지름 감소
- 각도 증가
- 반지름 복구

